Stock Market Prediction System

Using a LSTM and GRU Hybrid Neural Network

Directed Studies Project Report

Applied Machine Learning | Time-Series Forecasting

Institution: University of the Fraser Valley

Under Guidance of Omer Waqar
Student Name: Balkar Singh

Winter - 2026

Abstract

Forecasting stock market behavior remains one of the most challenging problems in financial data science due to the inherent volatility, noise, and non-stationarity of financial time-series data. This project explores the development of a predictive deep learning pipeline designed to forecast short-term market behavior. Through a structured, three-phase methodological approach, the project evaluates the effectiveness of recurrent neural network architectures—progressing from a baseline Long Short-Term Memory (LSTM) network to a sophisticated Bidirectional LSTM (BiLSTM) and Gated Recurrent Unit (GRU) hybrid model.

Initial experimentation in Phase 1 and a multi-stock Phase 2 revealed a critical limitation in traditional time-series forecasting: models tasked with predicting raw, absolute closing prices naturally mimicked naive persistence baselines, capturing generic trends but suffering from weak directional accuracy (averaging ~50.9%). To overcome this, the final Phase 3 architecture fundamentally reformulated the problem domain. The BiLSTM-GRU hybrid model was transitioned to forecast normalized next-day percentage returns utilizing a robust, 7-dimensional engineered feature space (incorporating moving averages, return volatility, and volume change).

Evaluated with strict chronological train-test splits on highly volatile assets (e.g., TSLA), the Phase 3 return-based model achieved significant improvements in both stability and practical utility. It yielded robust regression metrics ($9.35 MAE, 0.97 R^2) and outpaced naive "Always-UP" baselines by achieving a directional accuracy of 72.45%. Ultimately, this research demonstrates that while deeper architectures assist with feature extraction, reformulating the prediction target away from scale-dependent prices toward normalized returns is the most critical factor in developing actionable, predictive financial models.

TABLE OF CONTENTS

1. Introduction **5**
2. Background Study and Literature Review **6**
2.1 Background Work 6
2.2 Evolution of Forecasting Models 7
2.3 Machine Learning Based Approaches 7
2.4 Deep Learning Models for Financial Time-Series 7
2.5 Applications in Other Time-Series Domains 9
2.6 Evaluation Metrics and Research Gap 9
3. Problem Definition **10**
3.1 Price Prediction vs Direction Prediction 10
3.2 Initial Approach 10
3.3 Key Challenges Encountered 11
3.4 Final Problem Formulation 11
4. Data Collection and Preprocessing **13**
4.1 Data Source: Yahoo Finance API 13
4.2 Automated Data Preparation Pipeline (Data Cleaning) 15
4.3 Date Formatting and Chronological Integrity 16
4.4 Rigorous Train/Test Splitting 16
4.5 Feature Normalization and Leakage Prevention 17
4.6 Sequence Generation via Sliding Windows 18
5. Feature Engineering **19**
5.1 Price Relativization: Daily Returns 19
5.2 Trend Identification: Simple Moving Averages (SMA) 19
5.3 Risk Measurement: Rolling Volatility 19
5.4 Market Activity: Volume Change 20
5.5 Synthesized Feature Space and Visualization 20
6. Model Development and Training **22**
6.1 Architectural Evolution 22
6.2 Phase 1 LSTM Baseline Model 22
6.3 Phase 2 BiLSTM + GRU Hybrid Model 25
6.4 Phase 3: Final Hybrid LSTM–GRU Model (Return-Based Prediction) 31
7. Evaluation Metrics and Future Forecasting **37**
7.1 Regression Evaluation Metrics 37
7.2 Directional Accuracy 37
7.3 Comparison between Phase 1, 2 and 3 38
7.31 Phase 1 Evaluation 39
7.32 Phase 2 Evaluation 40
7.33 Phase 3 Evaluation 45
8. Project Reflection **56**
A. Key Project Successes (What Worked Well) 56

B. Technical Challenges Encountered 57
C. Technical Skills Gained and Lessons Learned 57
9. Conclusion and Future Work 59
Future Work 59
References 61
10. Appendix 64
Technology Stack 64
Dataset Description 64
Feature Engineering 64
Model Configuration 64
Evaluation Metrics 65
Experimental Setup 65
Project Repository 65

1. Introduction

Predicting stock market behavior is one of the most challenging problems in financial data science. Financial markets are inherently complex and are influenced by a wide range of factors, including economic indicators, investor sentiment, global events, and unpredictable market dynamics. As a result, stock price movements often exhibit high levels of noise and volatility, making accurate forecasting extremely difficult [1].

Despite these challenges, advances in machine learning, particularly deep learning techniques have shown promising capabilities in modeling time-series data. Recurrent neural networks (RNNs), especially Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) architectures, are designed to capture temporal dependencies and patterns in sequential data, making them well-suited for financial forecasting tasks [9][22].

Purpose and Motivation

The primary motivation for this project arises from the limitations of traditional statistical models (such as ARIMA) in handling the nonlinear and highly volatile nature of financial data. While these early models assume linear dependencies, deep learning techniques offer the capability to identify unstructured, subtle temporal patterns [4][29].

The goal of this project is to build and evaluate a hybrid deep learning-based system capable of predicting stock market behavior using historical data. Moving beyond traditional approaches that target solely exact price prediction, this project explores the prediction of directional price movement (returns). This shift in formulation allows for a more holistic evaluation of model performance and highlights practical actionability in real-world trading contexts.

Project Objectives

To address these goals, the project sets out to achieve the following core objectives:

- **Develop a Deep Learning Pipeline**: Construct an advanced, predictive neural network architecture incorporating LSTM and GRU layers.
- **Problem Formulation Evolution**: Explore and compare direct price prediction against return-based directional prediction to improve model stability.
- **Feature Engineering**: Implement advanced financial feature engineering (e.g., rolling averages, volatility, momentum) to enrich raw data and enhance predictive capability.
- **Robust Evaluation**: Benchmark the final model against a baseline configuration using stringent metrics including Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Directional Accuracy.
- **Analyze Generalization**: Explore the practical limitations, scaling challenges, and real-world applicability of deploying these models across diverse assets.

2. Background Study and Literature Review

2.1 Background Work

The Efficient Market Hypothesis (EMH) provides a foundational theoretical framework for understanding stock market behaviour. According to Fama [1], asset prices reflect available information, implying that consistently achieving abnormal returns through historical price analysis alone is inherently difficult. EMH is commonly classified into three forms: weak, semi-strong, and strong, each representing a different level of market efficiency based on the type of information incorporated into asset prices. In the weak form, all historical price information is assumed to be already reflected in current prices, which directly challenges the effectiveness of time-series-based stock prediction models.

This theoretical perspective highlights the inherent uncertainty and stochastic nature of financial markets and explains why stock price prediction remains one of the most difficult problems in applied machine learning. Financial time-series data is influenced by numerous interacting factors, including macroeconomic conditions, geopolitical events, investor sentiment, behavioural biases, and unexpected market shocks. These influences introduce significant noise and randomness into price movements, making it difficult to distinguish meaningful predictive patterns from short-term fluctuations.

In addition to being noisy, stock markets are characterised by high volatility, non-stationarity, and complex nonlinear dynamics. Non-stationarity means that the statistical properties of the data, such as mean and variance, may change over time, violating assumptions made by many traditional forecasting models [2], [3]. Furthermore, market behaviour often exhibits regime shifts, where patterns that appear stable in one period may fail completely in another. These properties make it especially challenging to develop predictive models that generalise reliably across different time periods and market conditions.

As a result, traditional linear forecasting methods often struggle to capture real-world market behaviour effectively. Early approaches relied on statistical techniques such as autoregression, moving averages, and differencing, which assume relatively stable relationships between past and future values [2], [3]. Although these methods remain valuable for foundational analysis, they are limited in their ability to model the nonlinear relationships and temporal complexity found in real financial data. Over time, advances in computational power and data availability have enabled the transition from classical statistical models to machine learning and deep learning approaches.

Time-series forecasting therefore plays a central role in stock price prediction. Core forecasting concepts such as trend, seasonality, and stationarity remain important even in modern machine

learning pipelines [2], [3]. Trend components capture long-term directional movement, while transformations such as differencing or return calculation are often used to stabilise the series before modelling. In this sense, successful machine learning in finance still depends heavily on proper preprocessing and problem formulation, not merely on model complexity.

Despite the theoretical difficulty implied by EMH, empirical research suggests that financial markets are not perfectly efficient in practice [32]. Short-term inefficiencies, delayed information diffusion, and behavioural effects may create exploitable local patterns within historical data. This motivates the use of machine learning methods, which are capable of learning subtle nonlinear relationships from large volumes of sequential data. In particular, deep learning architectures such as LSTM and GRU networks are well-suited for this task because they are specifically designed to model temporal dependencies in time-series data [8] [10].

2.2 Evolution of Forecasting Models

Classical Statistical Models Early stock market forecasting research relied heavily on statistical time-series models. The Box-Jenkins methodology, particularly the Autoregressive Integrated Moving Average (ARIMA) model, has been widely used for modelling linear dependencies in financial time series. Ariyo et al. [4] demonstrated the application of ARIMA for stock price prediction and reported reasonable short-term forecasting performance. Despite their simplicity and interpretability, ARIMA models assume linearity and stationarity, limiting their ability to capture the complex nonlinear patterns present in real stock market data. These limitations directly motivated the exploration of machine learning and deep learning alternatives.

2.3 Machine Learning Based Approaches

With advances in computational power and data availability, machine learning-based approaches emerged as strong alternatives to traditional statistical models. A comprehensive survey by Atsalakis and Valavanis [5] highlights the effectiveness of neural networks and hybrid learning models in capturing nonlinear patterns and complex market dynamics. Patel et al. [6] proposed a fusion-based framework for stock market index prediction, demonstrating that integrating multiple learning algorithms can significantly enhance forecasting accuracy. Their findings emphasise the importance of effective feature engineering and model combination strategies, a principle directly applied in this project through ten scale-independent technical indicators [31][33]. Lin et al. [7] further demonstrated that deliberate feature selection significantly influences prediction accuracy, reinforcing the value of engineered inputs over raw price data.

2.4 Deep Learning Models for Financial Time-Series

Deep learning has significantly advanced stock price prediction by enabling automatic feature learning from raw data. LeCun et al. [8] provide the foundational overview of deep learning's ability to model complex nonlinear relationships across domains, establishing the theoretical basis for applying neural architectures to financial time-series data.

- **Long Short-Term Memory (LSTM) Networks**

  Among deep learning models, Long Short-Term Memory (LSTM) networks have received particular attention due to their ability to model sequential data with long-range dependencies. Unlike standard recurrent neural networks, LSTM units contain an input gate, forget gate, and output gate that selectively retain or discard information over many time steps, solving the vanishing gradient problem. Fischer and Krauss [9] demonstrated that LSTM networks outperform traditional machine learning models in financial market prediction tasks, and Moghar and Hamiche [10] further confirmed improved forecasting accuracy using LSTM-based networks for stock price prediction. A comparison of CNN, LSTM, and ANN architectures by Wen [11] showed that deep learning architectures generally achieve superior predictive performance on financial data, with LSTM outperforming feedforward alternatives.

- **Bidirectional LSTM (BiLSTM)**

  A standard LSTM processes the input sequence in only one direction oldest to newest, potentially missing the context provided by more recent patterns when interpreting earlier time steps. The Bidirectional LSTM addresses this by processing the sequence in both the forward and backward directions simultaneously, concatenating the hidden states from both passes. In the context of stock prediction using a 90-day lookback window, this gives the model simultaneous access to long-term historical trends and the most recent price momentum, producing a richer representation than a unidirectional LSTM. This project adopts a Bidirectional LSTM as the first layer of its hybrid architecture for precisely this reason [24].

- **Gated Recurrent Units (GRU) and Hybrid Architectures**

  Gated Recurrent Units (GRU) offer a computationally lighter alternative to LSTM by combining the forget and input gates into a single update gate. This simplification reduces trainable parameters while achieving comparable performance on many sequential tasks, particularly where short-term dependencies are more important than very long-range memory. In stock price prediction, GRU layers are especially effective at detecting short-term momentum shifts and sharp price reversals. Wen [11] confirmed that recurrent architectures consistently outperform feedforward networks on financial data.

The combination of LSTM and GRU layers into a hybrid architecture has been validated in recent literature. Farhadi et al. [12] proposed and evaluated a hybrid LSTM-GRU model for stock price prediction, reporting improved accuracy and training stability compared to either architecture alone. Their results confirm that the two-stage design long-range context extraction followed by short-term feature sharpening produces a more robust prediction model. This

project extends their work by replacing the standard LSTM with a Bidirectional LSTM and adding a second GRU layer, further deepening the feature extraction pipeline while applying L2 regularisation to prevent overfitting across ten diverse stocks.

2.5 Applications in Other Time-Series Domains

Advanced forecasting models have been successfully applied beyond financial markets. Abdelsalam et al. [13] utilised gradient boosting for energy production forecasting, demonstrating the adaptability of machine learning models to complex temporal data. In traffic forecasting, Khan et al. [14] adopted hybrid architectures integrating graph-based learning with recurrent neural networks to capture both temporal dynamics and spatial interactions. Devi et al. [15] demonstrated that deep learning methods can be adapted effectively across temporal resolutions for energy consumption forecasting in smart home environments. These cross-domain findings consistently support the conclusion that hybrid recurrent models outperform single-architecture approaches on time-series prediction tasks, providing additional validation for the architectural choices made in this project.

2.6 Evaluation Metrics and Research Gap

Accurate evaluation of stock price prediction models is critical for meaningful comparison. A study in 2014 [16] analyzed commonly used error metrics, such as Root Mean Square Error (RMSE) and Mean Absolute Error (MAE), emphasizing that each metric captures different aspects of prediction error. RMSE penalizes large errors more heavily, while MAE provides a more robust measure of average performance. Another study in 2018 [17] highlighted challenges associated with validating time-series forecasting models, particularly the improper use of standard cross-validation techniques, and emphasized the need for time-aware evaluation strategies that respect the temporal structure of financial data.

Despite significant progress, several research gaps remain. Existing studies often employ different datasets, evaluation metrics, and validation strategies, making direct comparison difficult [18][29]. Additionally, many deep learning approaches prioritize accuracy improvements without adequately addressing generalization and robustness issues. These gaps motivate the development of a structured stock market price prediction system that integrates proper preprocessing, LSTM-based modeling, and rigorous evaluation using established performance metrics.

3. Problem Definition

Stock market prediction can be approached from multiple perspectives, with the two most common formulations being **price prediction (regression)** and **direction prediction (classification)**. Each approach serves a different purpose and presents its own set of challenges [32].

3.1 Price Prediction vs Direction Prediction

Price prediction focuses on estimating the exact future value of a stock, typically the next day's closing price. This is formulated as a regression problem, where the model outputs a continuous numerical value [6]. Accurate price prediction is useful for portfolio valuation, risk management, and long-term forecasting[.

However, predicting exact prices is extremely difficult due to the inherent noise and volatility in financial markets. Even small deviations in prediction can lead to large errors when evaluated using metrics such as RMSE. Additionally, price prediction models often struggle with scale sensitivity, where stocks with higher prices introduce larger numerical errors.

In contrast, direction prediction focuses on determining whether the stock price will move **up or down**, making it a binary classification problem [20]. This formulation is often more practical in real-world trading scenarios, as many trading strategies depend primarily on correctly identifying market direction rather than exact price values.

This project explores both approaches and evaluates their effectiveness. While price prediction provides detailed numerical forecasts, direction prediction offers more actionable insights, especially in short-term trading contexts.

3.2 Initial Approach

The initial objective of this project was to build a **multi-stock deep learning model** capable of predicting stock prices using a hybrid Bidirectional LSTM and GRU architecture. The idea was to train a single model across multiple stocks from different sectors, leveraging shared patterns in financial time-series data.

This approach involved:

- Using raw price-based features
- Training a unified model across multiple stocks
- Predicting next-day closing prices directly

However, during implementation and experimentation, **several challenges emerged that significantly affected model performance**.

3.3 Key Challenges Encountered

- **Cross-Stock Scaling Discrepancies**: Stocks naturally trade at vastly different magnitudes (e.g., a $40 stock versus a $900 stock). Using a shared normalization scaler across all assets completely distorted the data representation, causing the optimizer to heavily bias its gradient updates toward the higher-priced stocks.
- **Poor Generalization**: While the multi-stock concept was theoretically appealing, the model struggled to generalize across disparate sectors. Volatility profiles and market behaviors of an electric vehicle stock (e.g., TSLA) differ substantially from established software companies (e.g., MSFT). The shared model effectively averaged out these nuances, yielding inconsistent predictive reliability
- **Instability of Absolute Price Prediction**: Attempting to force the model to output raw, absolute prices natively resulted in high sensitivity to market noise. The model produced over-smoothed predictions that severely lagged behind sharp, sudden price reversals, leading to unacceptably high Root Mean Squared Errors.
- **Monotonic Trend Bias**: During multi-step, recursive forecasting horizons (e.g., predicting two weeks out), the model suffered from compounding error accumulation. This resulted in predictions taking on a monotonic trend (a perfect, straight line moving endlessly up or down) rather than demonstrating realistic day-to-day market fluctuations.

3.4 Final Problem Formulation

To overcome these obstacles, the problem was fundamentally reformulated. The objective shifted from predicting absolute stock prices across multiple assets, to predicting normalized daily stock returns for individual assets.

Instead of asking the model, "**What will the price be tomorrow?**" the model was trained to answer, "**What percentage will the stock move tomorrow**?"

This evolution offered substantial mathematical advantages. Returns are stationary, inherently scale-independent, and effectively neutralize the magnitude differences between high-priced and low-priced stocks [26][29]. The final pipeline structure emerged as follows:

- Uses **60-day lookback windows**
- Predicts **next-day returns**
- Converts returns into **price predictions**

- Evaluates performance using:
    - MAE, RMSE (price accuracy)
    - Directional Accuracy (trend prediction)

4. Data Collection and Preprocessing

The foundation of any robust machine learning pipeline lies in the quality, structure, and treatment of its data. Because time-series neural networks are highly sensitive to chronological integrity and scale, raw stock market data cannot be fed directly into a deep learning model. This section details the data acquisition process and the rigorous preprocessing pipeline utilized to ensure model reliability and prevent data leakage.

4.1 Data Source: Yahoo Finance API

Historical stock market data was programmatically collected using the **Yahoo Finance API** via the Python **yfinance** library [30]. This approach allows for automated, reproducible data retrieval with consistent formatting across multiple assets.

For the scope of this project, approximately ten years of daily trading data was collected. Utilizing a decade-long view is mathematically better, as it exposes the model to a wide spectrum of market. The full set of project datasets was stored in **Google Drive** and later accessed within the **Google Colab** environment by mounting the drive to the notebook session.

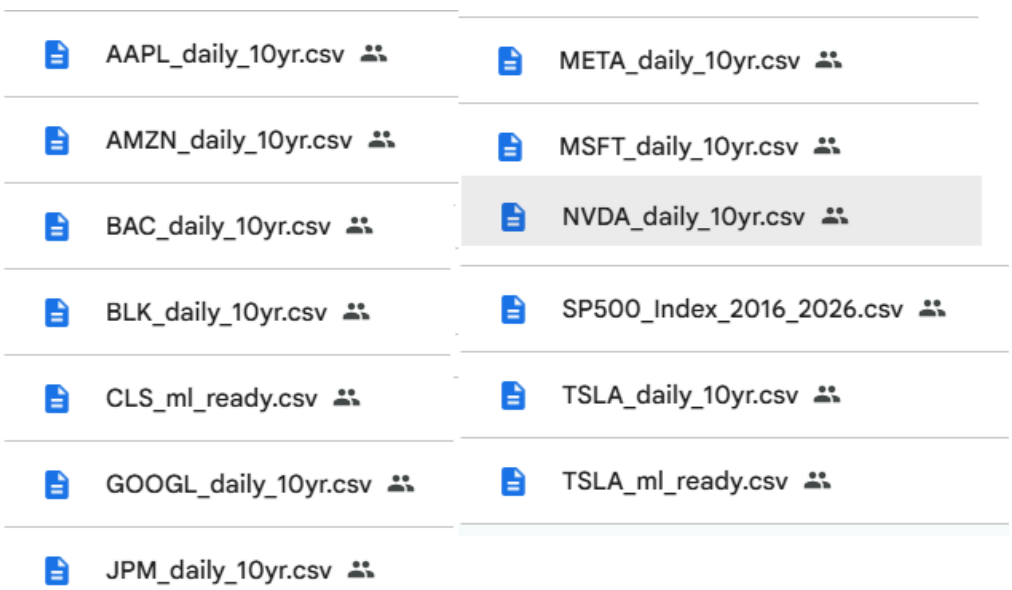


Figure 4.1 shows multiple CSV files containing approximately 10 years of daily historical stock data for different companies. These datasets formed the raw input for the project and supported both single-stock and multi-stock experiments.

The raw dataset included standard trading attributes: Date, Open, High, Low, Close, Adjusted Close, and Volume. For the primary target baseline, the Adjusted Close price was selected, as it accurately reflects true asset value by mathematically accounting for historical corporate actions such as stock splits and dividend distributions [26].

```
df.head()
```

	Date	Open	High	Low	Close	Adj Close	Volume
0	2016-01-04	15.381333	15.425333	14.600000	14.894000	14.894000	102406500
1	2016-01-05	15.090667	15.126000	14.666667	14.895333	14.895333	47802000
2	2016-01-06	14.666667	14.670000	14.398667	14.602667	14.602667	56686500
3	2016-01-07	14.279333	14.562667	14.244667	14.376667	14.376667	53314500
4	2016-01-08	14.524000	14.696000	14.051333	14.066667	14.066667	54421500

Figure 4.2 (df.head() showing the raw columns) for TSLA raw data fetched from Yahoo Finance.

Figure 4.3 shows the raw adjusted closing price of **Tesla (TSLA)** over the period from **2016 to 2026**. The chart illustrates the highly volatile nature of stock market data, with periods of rapid growth, sharp declines, and frequent fluctuations over time. This visual highlights one of the core challenges of stock price prediction, as the model must learn from data that is noisy, non-stationary, and strongly influenced by changing market conditions.

4.2 Automated Data Preparation Pipeline (Data Cleaning)

To ensure absolute consistency and reproducibility across multiple datasets, a dedicated, automated data preparation script was engineered. Rather than cleaning data manually in scattered notebooks, this centralized pipeline was deployed to ingest raw CSV files, perform rigorous data scrubbing, generate features, and output definitive "ML-ready" datasets.

The script incorporates an enterprise-standard logging framework (tracking success statuses, error handling, and directory creation) and executes the following core data structuring operations:

- **Column Scrubbing and Validation**: Verifies the presence of required attributes and strips formatting inconsistencies.
- **Duplicate Elimination**: Scans the Date column to drop duplicated trading days that inherently pollute time-series consistency.
- **Feature Appendin**g: Automatically computes derived financial indicators (e.g., Moving Averages, Return Volatility, Volume Change) concurrently while retaining raw price attributes.
- **Target Calculation**: Mathematically derives the next-day Target label for classification (identifying whether the next day's Adjusted Close moves higher than the current day).
- **Discontinuity Removal**: Systematically drops #NaN rows that are naturally generated during rolling window computations.

```
df.head()
```

```
Shape: (2511, 14)
['Date', 'Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume', 'Return', 'MA10', 'MA20', 'MA50', 'Volatility10', 'Volume_Change', 'Target']
```

	Date	Open	High	Low	Close	Adj Close	Volume	Return	MA10	MA20	MA50	Volatility10	Volume_Change	Target
0	2016-03-15	14.284667	14.598000	14.100000	14.556000	14.556000	47707500	0.014827	13.652667	12.799767	12.685773	0.019189	-0.217724	1
1	2016-03-16	14.533333	14.838667	14.468000	14.795333	14.795333	52750500	0.016442	13.876600	12.977267	12.683800	0.019093	0.105707	1
2	2016-03-17	14.764667	15.233333	14.666667	15.092000	15.092000	56743500	0.020051	14.080867	13.175967	12.687733	0.017464	0.075696	1
3	2016-03-18	15.273333	15.632000	15.204000	15.516000	15.516000	70677000	0.028094	14.292200	13.396500	12.706000	0.017547	0.245552	1
4	2016-03-21	15.689333	15.992000	15.666667	15.888000	15.888000	79617000	0.023975	14.512400	13.598433	12.736227	0.017681	0.126491	0

Figure 4.3. Example of the processed dataset after the automated data preparation pipeline, showing original stock attributes together with engineered features and the target variable used for machine learning.

The execution of this script results in a sanitized, structured dataset saved directly to a dedicated **/PreparedData** directory, serving as the pristine source of truth for the deep learning models.

4.3 Date Formatting and Chronological Integrity

Following the automated cleaning, an integral step is ensuring time-series integrity. The **Date** column generated by the pipeline is explicitly converted into standard Python **datetime** objects and subjected to strict secondary sorting in ascending chronological order.

Unlike standard tabular machine learning tasks where data can be randomly shuffled to reduce variance, time-series forecasting mandates strict temporal preservation. Shuffling time-series data destroys the fundamental sequential **"memory"** dependencies that **LSTM and GRU** networks rely upon to function [17].

4.4 Rigorous Train/Test Splitting

Standard machine learning models often utilize randomized cross-validation (such as K-Fold splits). However, applying randomized splitting to time-series data inadvertently introduces data leakage where the model gains access to "future" information during the training phase, artificially inflating its apparent accuracy [17].

To circumvent this, the data was strictly divided using an 80/20 Chronological Split. The first 80% of the timeline was exclusively allocated for the training set, while the most recent 20% of the timeline was partitioned sequentially as the unseen testing set.

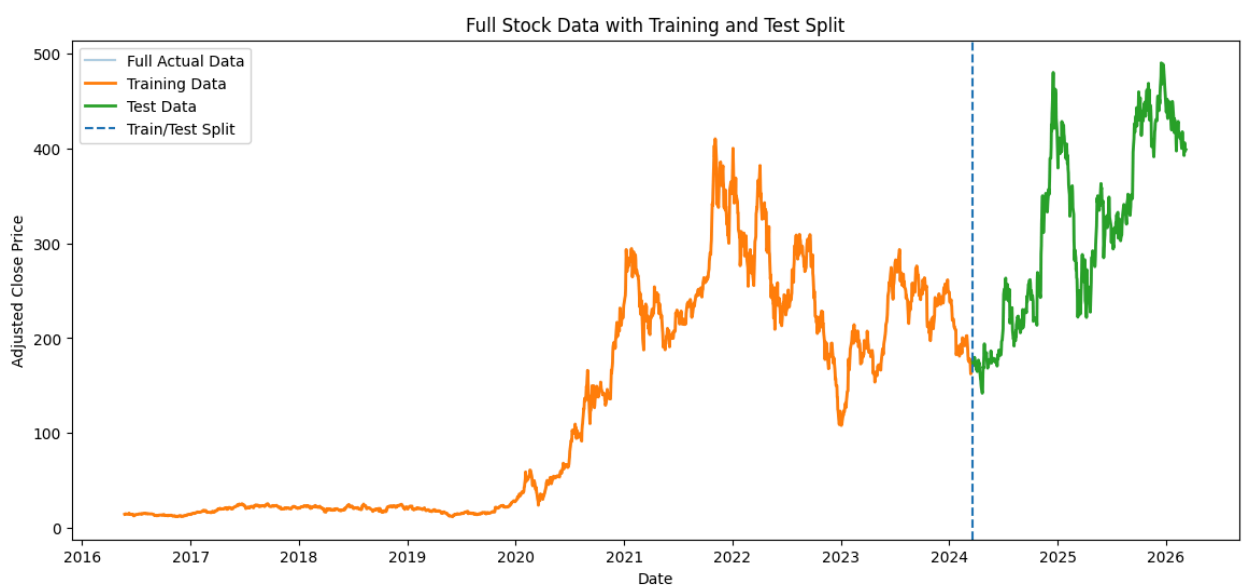


Figure 4.4 Chronological train-test split used for time-series forecasting. The training set contains the earlier 80% of observations, while the testing set contains the remaining 20%, preserving temporal order and preventing data leakage (The example shows TSLA Stock data).

4.5 Feature Normalization and Leakage Prevention

Financial data inherently contains vast differences in magnitude. For instance, daily trading volume is measured in the tens of millions, while a normalized daily return is typically a fraction of a percent. Passing unscaled data into a neural network results in heavily skewed gradient calculations, as large-magnitude variables heavily dominate the activation weights.

To address this, all input variables were normalized into a uniform [0, 1] range via a MinMaxScaler algorithm [30]. Standardizing the features ensures that the Bidirectional LSTM and GRU layers distribute weight updates symmetrically, significantly reducing convergence time and enhancing stability.

Preventing Leakage During Scaling: A critical methodological constraint applied in this project was fitting the scaling algorithm exclusively on the training data. If the scaler were to analyze the entire dataset prior to the split, the maximum and minimum values of the "unseen" test set would mathematically influence the training inputs, causing indirect leakage. By fitting the scaler on the training distribution, those identical transformation rules were uniformly applied to the test set, ensuring completely realistic, out-of-sample forecasting conditions [17].

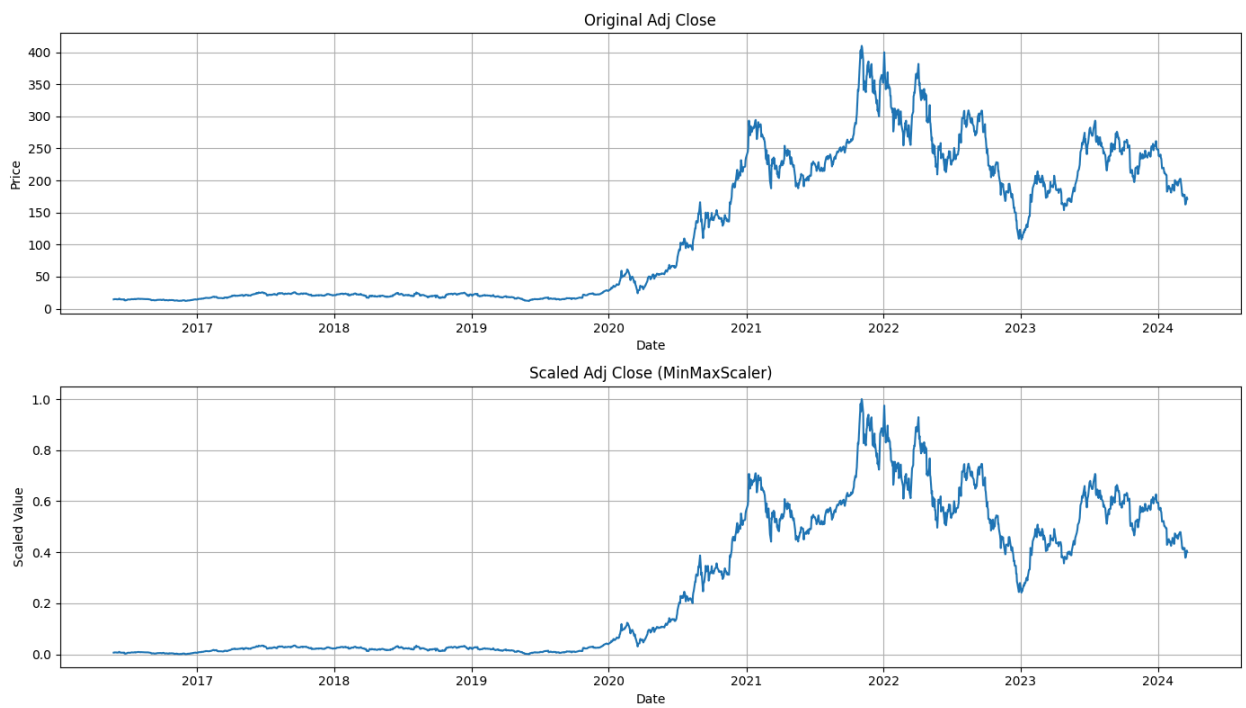


Figure 4.5 Before and after feature normalization using MinMaxScaler. The top plot shows the original adjusted closing price of TSLA, while the bottom plot shows the same feature scaled to the

[0,1][0,1][0,1] range. The normalization preserves the overall trend while reducing the magnitude differences, improving model training stability.

4.6 Sequence Generation via Sliding Windows

Recurrent neural networks require input data to be grouped into discrete sequential payloads. This was achieved using a sliding window technique.

A **60-day Lookback Window** was configured. For every training observation, the model is fed a multi-dimensional array encapsulating **60 consecutive trading days** of engineered features. The target label mapped to this specific sequence is the predicted movement of the **61st day** [22]. The window then slides forward by exactly one day, and the process is recursively repeated across the entire timeline. This structural design enables the network to isolate and study short-term momentum shifts leading directly up to the targeted event horizon.

5. Feature Engineering

Financial markets are driven by underlying dynamics such as momentum, trend strength, and trading volume. Feature engineering is the important process of mathematically extracting these latent signals from the raw data, thereby providing the model with a structured, multi-dimensional view of the market environment.

5.1 Price Relativization: Daily Returns

Raw price values are absolute and heavily scale-dependent. To neutralize scale and provide a standardized measurement of price movement, the Adjusted Close was transformed into a Daily Return metric.

Returns measure the percentage change in asset price from one trading session to the next [26]. By analyzing returns rather than absolute prices, the model becomes significantly more stable, as it learns to predict standardized percentage shifts rather than attempting to guess arbitrary numerical dollar values.

5.2 Trend Identification: Simple Moving Averages (SMA)

Financial time-series data is inherently chaotic, characterized by significant day-to-day fluctuations. To help the network distinguish actual macroeconomic trends from daily noise, three Simple Moving Averages were computed across different temporal windows:

MA10 (Short-Term Trend): Captures immediate price momentum and rapid reversals over a two-week trading period.

MA20 (Medium-Term Trend): Provides insight into the monthly baseline consensus.

MA50 (Long-Term Trend): Establishes the overarching macroeconomic trajectory of the asset [28].

The spatial convergence and divergence of these three distinct averages allow the network to inherently learn moving average crossover strategies, a proven metric for identifying bullish and bearish momentum shifts.

5.3 Risk Measurement: Rolling Volatility

Price movement alone does not fully encompass market context; the velocity of that movement is equally critical. To quantify risk and market uncertainty, a 10-day Rolling Volatility feature was engineered.

This metric calculates the rolling standard deviation of the daily returns over a 10-day window [26]. Spikes in this feature explicitly inform the neural network that the market is experiencing

localized shocks or erratic trading, allowing the model to dynamically adjust the confidence weights of its predictions during periods of high turbulence.

5.4 Market Activity: Volume Change

Price adjustments occurring on low trading volume often lack conviction and are prone to reversal. Conversely, large price swings accompanied by heavy trading volume indicate strong market consensus (either buying or selling pressure) [28]. To explicitly capture this dynamic, the raw volume data was converted into a Volume Change percentage. This metric allows the recurrent layers to correlate the magnitude of price movements with the underlying conviction of the broader market.

5.5 Synthesized Feature Space and Visualization

With these features successfully engineered, the final multidimensional tensor passed into the LSTM/GRU sliding window comprised the following attributes: Adjusted Close, Daily Return, MA10, MA20, MA50, Volatility10, and Volume Change.

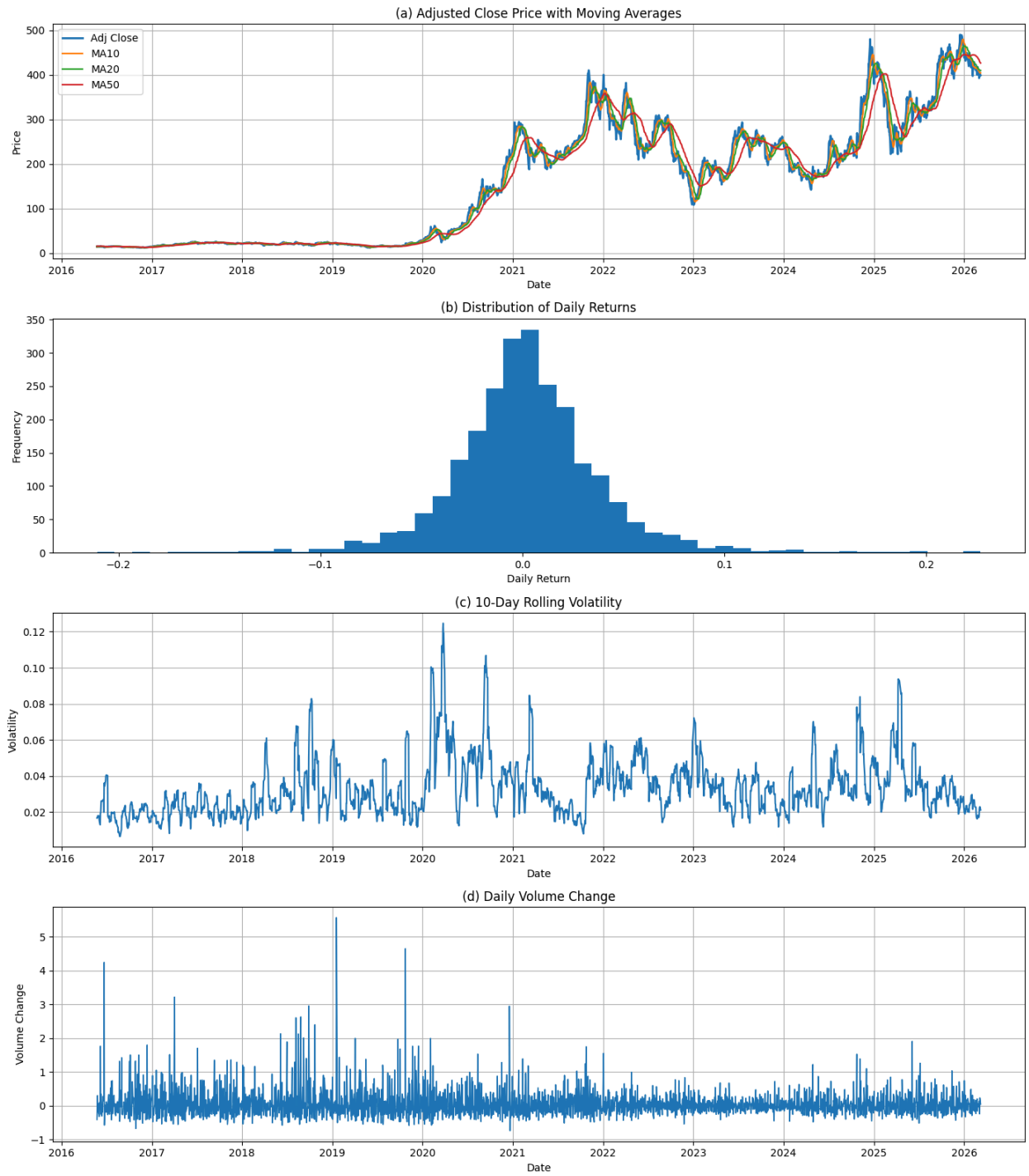


Figure 5 Feature engineering visualization for the final model. Panel (a) shows the adjusted closing price with short-, medium-, and long-term moving averages (MA10, MA20, MA50). Panel (b) shows the distribution of daily returns. Panel (c) presents the 10-day rolling volatility, and panel (d) shows daily volume change. Together, these plots illustrate the engineered features used to capture trend, momentum, volatility, and market activity. (the above graph represent 10yr TSLA stock data)

6. Model Development and Training

The core neural network architecture was not finalized in a single iteration; rather, it evolved through a structured, three-phase development methodology. By systematically testing baseline architectures and identifying their mathematical bottlenecks, the model was incrementally upgraded to handle the unique complexities of financial time-series data.

6.1 Architectural Evolution

The final model is a **hybrid deep learning architecture** that combines **Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) layers**, designed to predict next-day stock returns based on historical sequences of engineered financial features.

Unlike the initial model, which attempted to predict raw stock prices, the final model focuses on predicting returns. This transformation improves learning stability, reduces scale dependency, and allows the model to better capture relative price movements rather than absolute values.

Model development was carried out in three structured phases. In **Phase 1**, a **baseline LSTM model** was implemented to establish a reference point for performance. In **Phase 2**, the architecture was enhanced by introducing a **Bidirectional LSTM (BiLSTM) combined with a GRU layer** to better capture temporal dependencies in both forward and backward directions. Finally, in **Phase 3**, the model was refined into a **hybrid LSTM-GRU architecture**, which demonstrated the most stable performance and produced the best overall results [9][12][33].

This incremental and iterative approach enabled a controlled comparison between different model architectures under identical data conditions and evaluation metrics, ensuring that improvements were both measurable and justified. This section outlines the progressive architectural journey, culminating in the deployment of the final, highly optimized Phase 3 hybrid model.

6.2 Phase 1 LSTM Baseline Model

The first phase of the project was designed to establish a clear baseline for stock price forecasting using a simple recurrent neural network. At this stage, the goal was to test whether a standard Long Short-Term Memory (LSTM) model could learn meaningful temporal patterns from historical stock data before introducing additional complexity [9]. The experiment was conducted on a **single stock**, using Apple (AAPL), so that the behaviour of the model could be observed in a controlled and interpretable setting.

In this phase, the model was trained to predict the **next-day closing price** directly from historical closing-price sequences [10]. Only the raw closing price was used as input, without the addition

of engineered features such as returns, moving averages, or volatility indicators. This made Phase 1 a true baseline experiment, intended to provide an initial performance benchmark and reveal the limitations of a simple price-based forecasting approach.

Data Representation and Preparation

Historical daily stock data for **AAPL** was collected from Yahoo Finance and cleaned by removing missing values. The closing-price series was normalized using **MinMaxScaler** to ensure consistent scaling, which improves neural network training efficiency.

A **60-day lookback window** was used to convert the time series into supervised learning sequences, where the model uses the previous 60 days to predict the next day's price. The dataset was then divided into **80% training and 20% testing sets** using a chronological split to maintain temporal order and avoid data leakage.

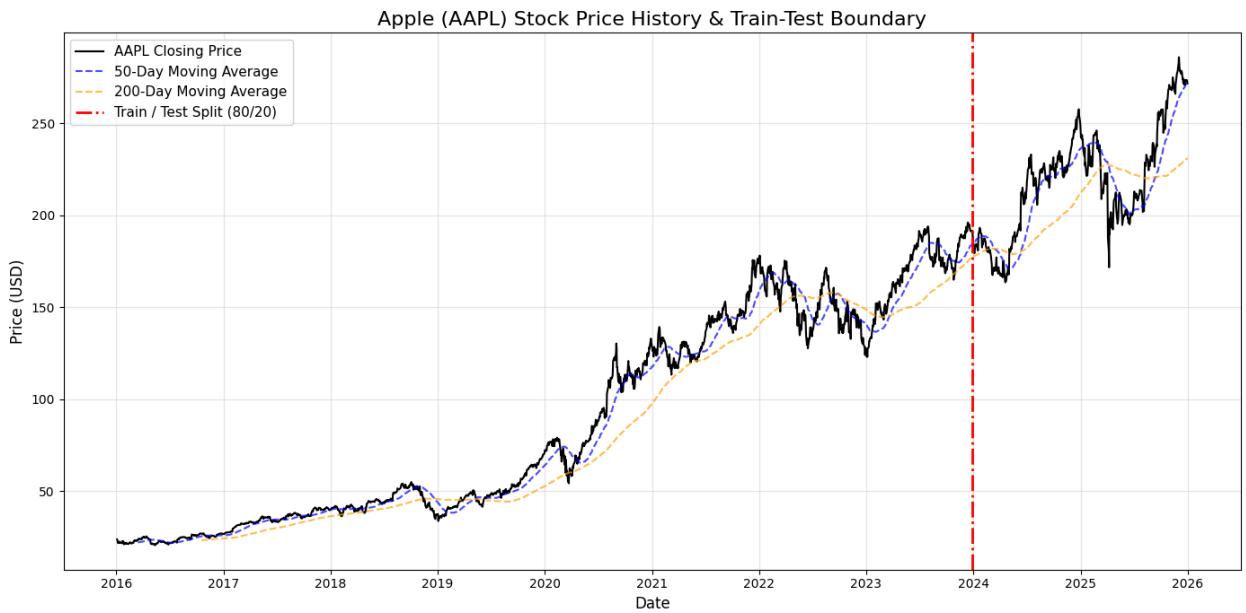


Figure 6.2 shows the AAPL (apple stock) price history with test/train split.

Model Architecture

The Phase 1 baseline used a simple Sequential LSTM network consisting of one recurrent layer, followed by dropout regularization and a dense output node.

Layer	Type	Units / Details
1	Input	Shape: (60, 1) → 60 timesteps, 1 feature
2	LSTM	50 units
3	Dropout	Rate: 0.2
4	Dense (output)	1 unit - predicted Close price

This architecture was intentionally kept simple so that it could serve as a reference point for later phases of the project.

Training Configuration and Callback

The model was compiled using the **Adam optimizer** and **Mean Squared Error (MSE)** loss, since the task in this phase was a regression problem involving the prediction of a continuous stock price value [8]. Training was performed for up to **30 epochs** with a **batch size of 32**, while monitoring validation loss. **Early stopping** was also applied to reduce overfitting and restore the best-performing weights.

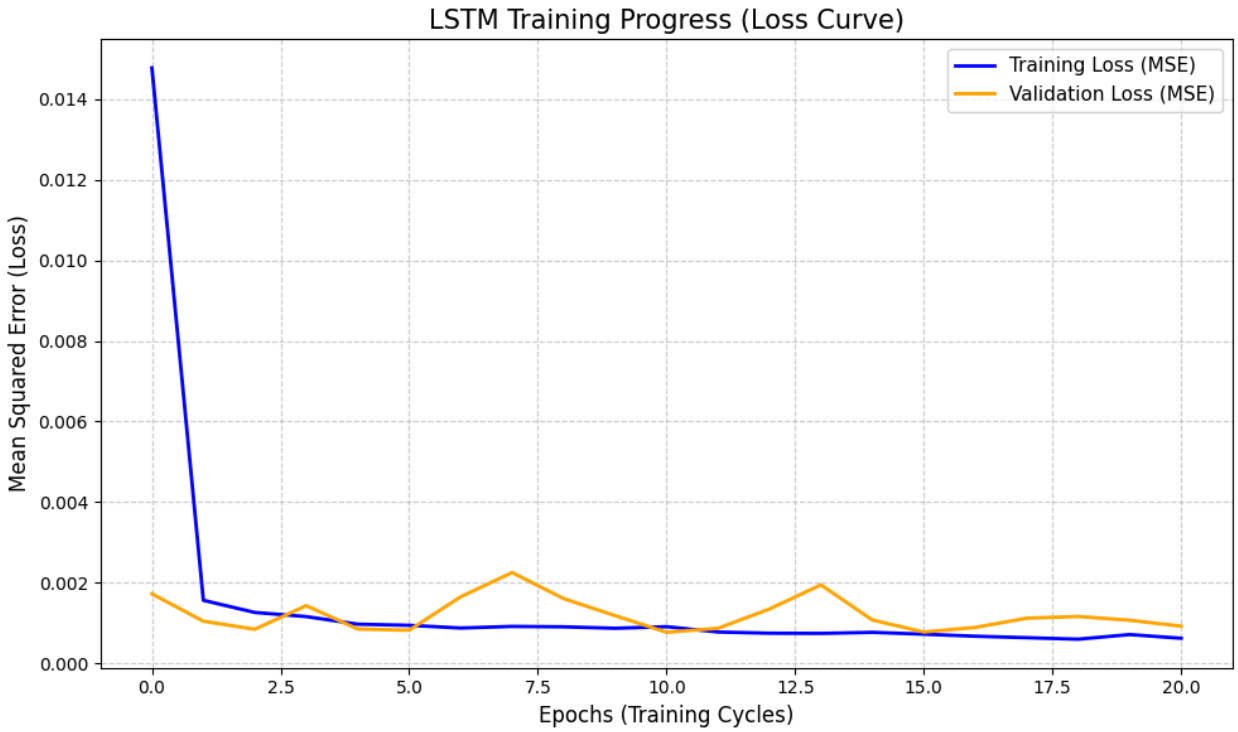


Figure 6.23 illustrates the training and validation loss over epochs. The model converges quickly, and the small gap between the curves indicates minimal overfitting and stable learning. Early stop at Epochs 20.

<u>What Was Learned from Phase 1</u>

Phase 1 was an important diagnostic step in the project. It confirmed that an LSTM-based model could extract some temporal structure from stock price data, but it also showed that a simple single-feature price-prediction setup was not sufficient for strong performance. This phase established the baseline against which the more advanced models in later phases were evaluated.

6.3 Phase 2 BiLSTM + GRU Hybrid Model

The second phase of model development introduced a hybrid architecture that replaces the stacked LSTM layers with a **Bidirectional LSTM (BiLSTM)** followed by a **GRU layer**, along with **Batch Normalization** to improve training stability.

<u>Motivation for the Hybrid Architecture</u>

A standard LSTM processes the input sequence in a single direction (from past to present). In contrast, a **Bidirectional LSTM** processes the sequence in **both forward and backward directions**, combining the hidden states from both passes [21].

This enables the model to interpret each time step with richer context, allowing it to better distinguish between:

- Temporary price fluctuations that recover
- Sustained trends indicating long-term movement

The **GRU layer** was introduced after the BiLSTM to further refine the learned representations.

This creates a two-stage feature extraction process that improves the model's ability to react to rapid market changes.

Batch Normalization was applied after the BiLSTM layer to normalize activations before they are passed to the GRU [8]. This is particularly important in deep learning models trained on **noisy financial data**, where instability can significantly impact performance.

Data Representation

```
  AAPL    rows:  2156  train: 1634  test:  432
  AMZN    rows:  2201  train: 1670  test:  441
  BAC     rows:  2198  train: 1668  test:  440
  BLK     rows:  2139  train: 1621  test:  428
  GOOGL   rows:  2169  train: 1645  test:  434
  JPM     rows:  2122  train: 1607  test:  425
  META    rows:  2212  train: 1679  test:  443
  MSFT    rows:  2096  train: 1586  test:  420
  NVDA    rows:  2330  train: 1774  test:  466
  TSLA    rows:  2318  train: 1764  test:  464

Stocks loaded          : ['AAPL', 'AMZN', 'BAC', 'BLK', 'GOOGL', 'JPM', 'META', 'MSFT', 'NVDA', 'TSLA']
Combined train shape   : (16648, 90, 18)
Combined val shape     : (2196, 90, 18)
Combined test shape    : (2197, 90, 18)
  → 16648 training sequences
  → 90 days lookback
  → 18 features per day
```

The combined training dataset was constructed by loading each of the ten stock CSV files individually, applying per-stock MinMax scaling, generating sliding-window sequences, and concatenating all sequences into a single unified training array.

Model Architecture

The Phase 2 hybrid model consisted of the following layers:

Layer	Type	Units / Details

1	Input	Shape: (60, 6) 60 timesteps, 6 features
2	Bidirectional LSTM	64 units each direction (128 total), return_sequences=True
3	BatchNormalization	Normalises layer activations
4	Dropout	Rate: 0.3
5	GRU	64 units, return_sequences=False
6	Dropout	Rate: 0.3
7	Dense	32 units, ReLU activation
8	Dense (output)	1 unit predicted Close price

Training Configuration and Callbacks

To allow a controlled comparison with Phase 1, the Phase 2 model was trained under broadly similar conditions, with adjustments made to account for the greater architectural complexity.

Parameter	Phase 1 (LSTM)	Phase 2 (Hybrid)
Optimiser	Adam	Adam

Learning rate	0.001	0.0005
Loss function	Huber	Huber
Batch size	32	16
Max epochs	100	100
Shuffle	True	True

The learning rate for Phase 2 was reduced from **0.001 to 0.0005** because the hybrid model contains more parameters and benefits from more controlled gradient updates during optimization. The batch size was also reduced to 16 in order to improve training stability.

Two callbacks were applied during training:

- **EarlyStopping (patience = 15)**
  monitored validation loss and stopped training when performance no longer improved, while restoring the best model weights.
- **ReduceLROnPlateau (patience = 7, factor = 0.5)**
  reduced the learning rate whenever validation loss plateaued, enabling finer convergence in later epochs.

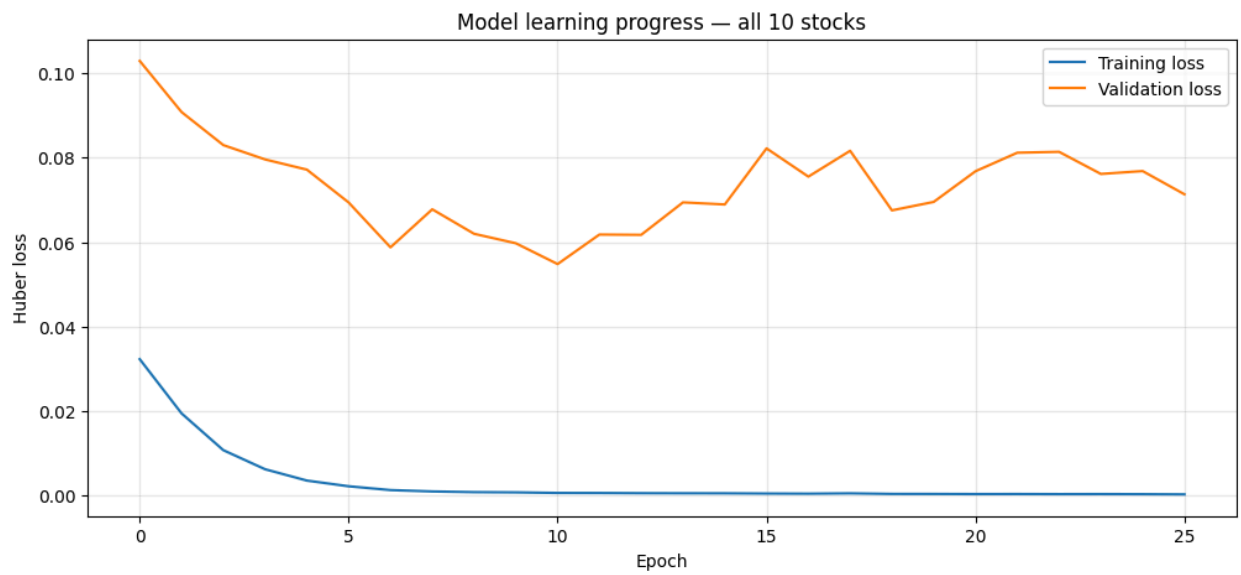


Figure 6.31 illustrates the training and validation loss for the multi-stock model. While training loss decreases to near zero, the higher and fluctuating validation loss indicates overfitting and limited generalization across different stocks.

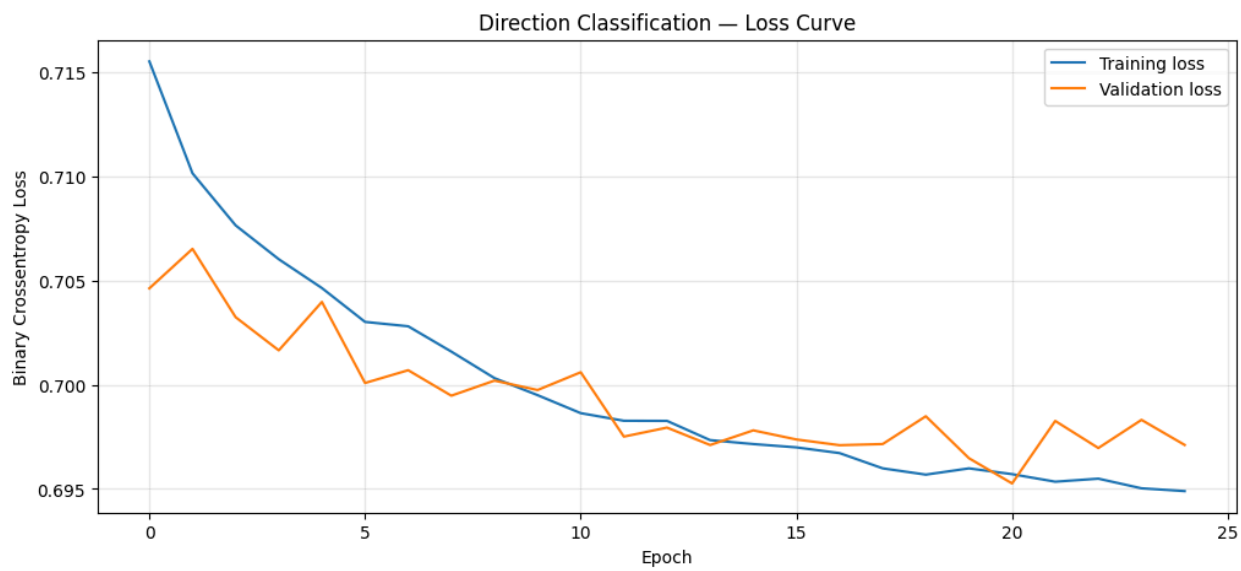


Figure 6.32 illustrates the training and validation loss for the direction classification model. The similar trends indicate stable learning with minimal overfitting, although the relatively high loss suggests limited predictive strength.

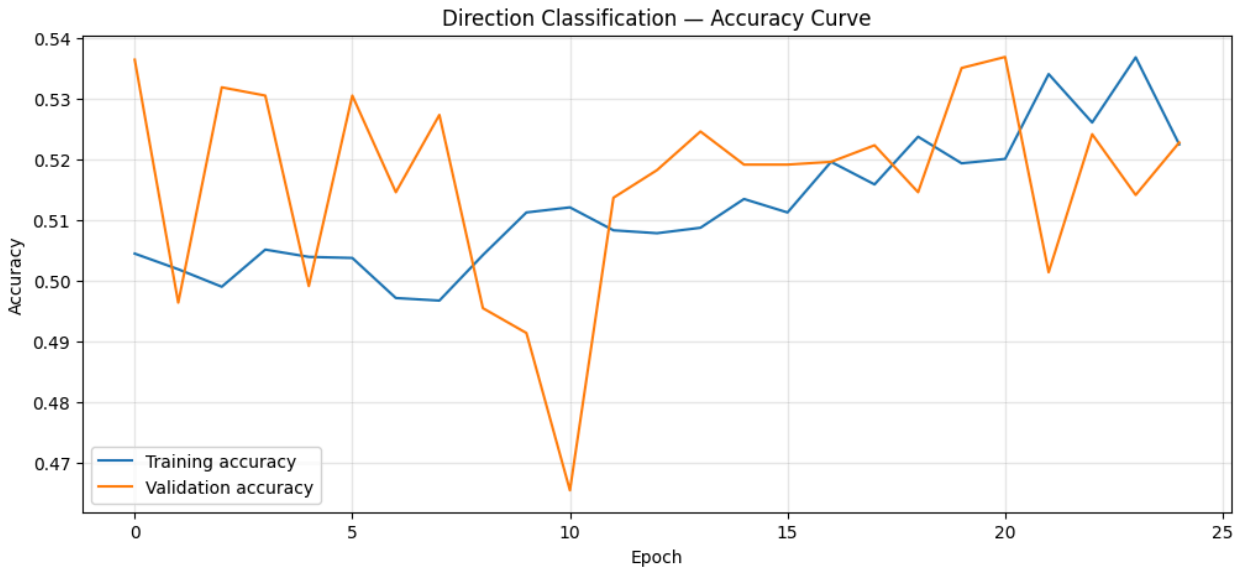


Figure 6.33 shows the training and validation accuracy for the classification model. Accuracy remains slightly above 50%, indicating modest improvement over random prediction but limited overall performance.

<u>What Was Learned from Phase 2</u>

Phase 2 represented an important architectural improvement over the baseline model. It demonstrated that adding richer features and a hybrid recurrent structure could improve the model's ability to represent stock market dynamics.

Although the BiLSTM–GRU model was more powerful than the baseline, later experimentation showed that performance still varied and that direct price prediction remained sensitive to noise, scale, and cross-stock inconsistency. This was especially important when the project expanded into the multi-stock setting, where different assets exhibited different volatility patterns and temporal behaviour.

These results indicate that direction classification alone is insufficient for reliable stock prediction which motivated the transition to Phase 3, where the focus shifted from raw price prediction to **return-based prediction** using a simplified but more stable hybrid LSTM–GRU design.

6.4 Phase 3: Final Hybrid LSTM–GRU Model (Return-Based Prediction)

The critical breakthrough of the project occurred in Phase 3. Synthesizing the lessons from the previous phases, the model was re-designed to combine regression precision with directional insight. Rather than attempting to classify raw logic or guess volatile dollar amounts, **the network was tasked with predicting normalized next-day percentage returns.**

Based on these findings, the final model was as a **single-stock, return-based forecasting system** using a hybrid **LSTM–GRU architecture** [12][25]. This change provided important improvement in the project, as it reduced scale sensitivity, improved stability, and produced stronger predictive performance.

Motivation for the Final Model

The final model was motivated by three main observations from earlier phases of the project.

First, **direct price prediction was unstable**. Raw stock prices are non-stationary, trend over time, and vary greatly in scale. These characteristics made the regression task more difficult and contributed to larger prediction errors [26][29].

Second, **multi-stock training reduces consistency**. When multiple stocks were combined into a single model, differences in volatility, price structure, and market behaviour made it harder for the model to generalize effectively across assets [29].

Third, **relative movement was more informative than absolute price value**. Predicting returns rather than raw prices allowed the model to focus on normalized market movement, which is more stable and more meaningful for short-term forecasting [30].

Data Representation and Input Features

The final model used a **60-day lookback window**, meaning that each training sample consisted of the previous 60 trading days of engineered features. Instead of relying only on raw closing price, the model was trained using the following seven input features:

- Adjusted Close
- Return
- MA10
- MA20
- MA50
- Volatility10
- Volume_Change

These features were selected to provide the model with information about trend, momentum, stability, and trading activity.

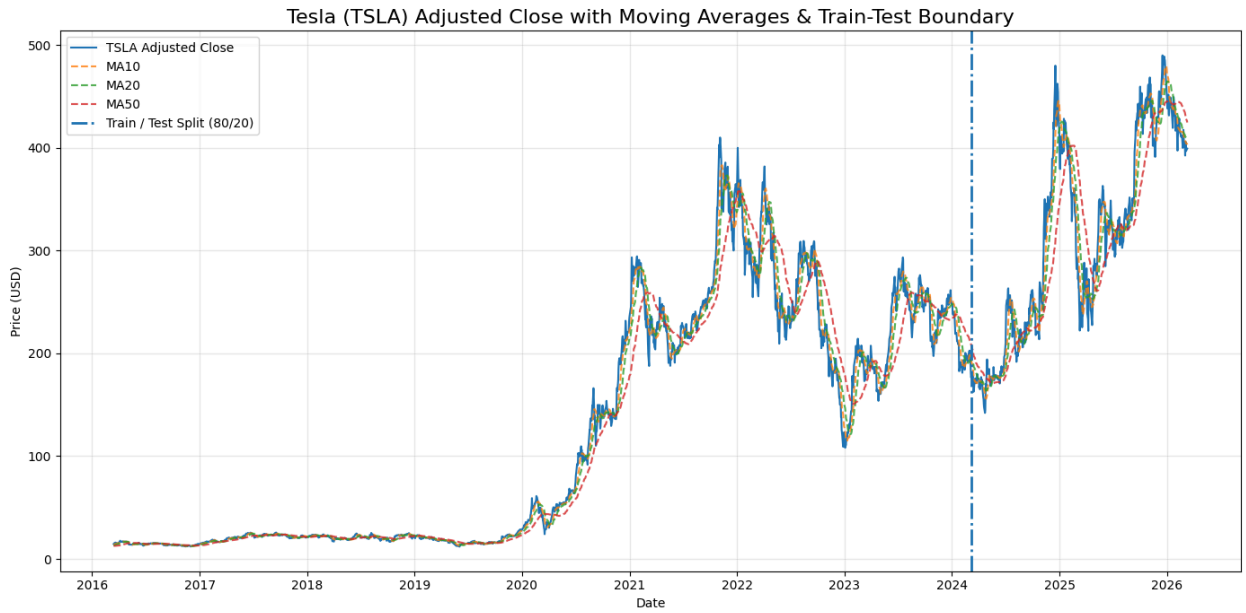


Figure 6.41 shows the adjusted closing price of TSLA together with the engineered moving averages (MA10, MA20, and MA50). The vertical line marks the 80/20 chronological train-test split. Represents volatility of TSLA stock and the trend information captured through feature engineering, which formed part of the input space for the final return-based LSTM–GRU model.

Model Architecture

Architecture and Hyperparameters (The TSLA Pipeline): Executed on TESLA stock 10 year stock data to capture volatility environments, the final architecture represents the culmination of the project's logic.

Layer	Type	Units / Details
1	Input	Shape: (60, 7) 60 timesteps, 7 features

2	LSTM	64 units, return_sequences=True
3	Dropout	Rate: 0.2
4	GRU	32 units
5	Dropout	Rate: 0.2.
6	Dense	32 units, ReLU activation
7	Dense (output)	1 unit predicted Close price

Training Configuration and Callbacks

The final model was compiled using the **Adam optimizer**, with **Mean Squared Error (MSE)** as the loss function and **Mean Absolute Error (MAE)** as an additional training metric.

The main training settings were:

- **Optimizer:** Adam
- **Loss Function:** MSE
- **Metric:** MAE
- **Epochs:** up to 30
- **Batch Size:** 32
- **Validation:** 10% of the training set
- **Callback:** EarlyStopping with patience = 5 and restore_best_weights=True

Early stopping was used to prevent overfitting. During training, the model monitored validation loss and stopped once no further improvement was observed for five consecutive epochs. The restore_best_weights=True setting ensured that the final saved model corresponded to the epoch with the best validation performance rather than the final training epoch.

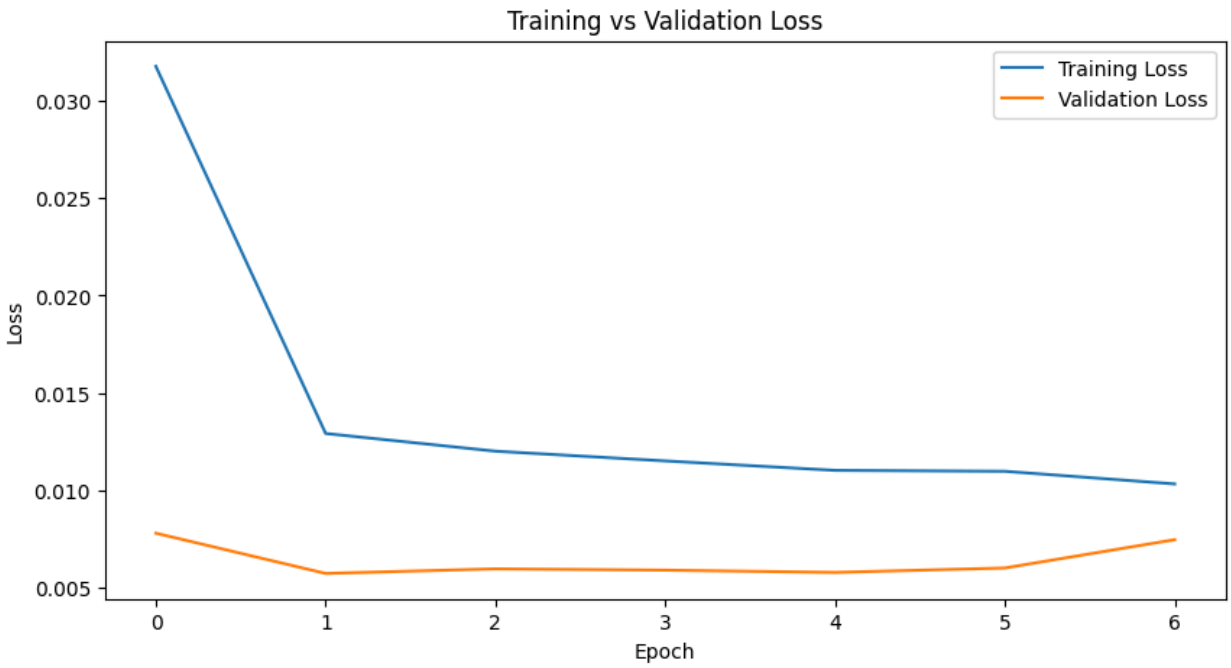


Figure 6.42 illustrates the training and validation loss for the Phase 3 model. Both curves converge quickly and remain close, indicating stable learning and minimal overfitting, with improved generalization compared to earlier phases.

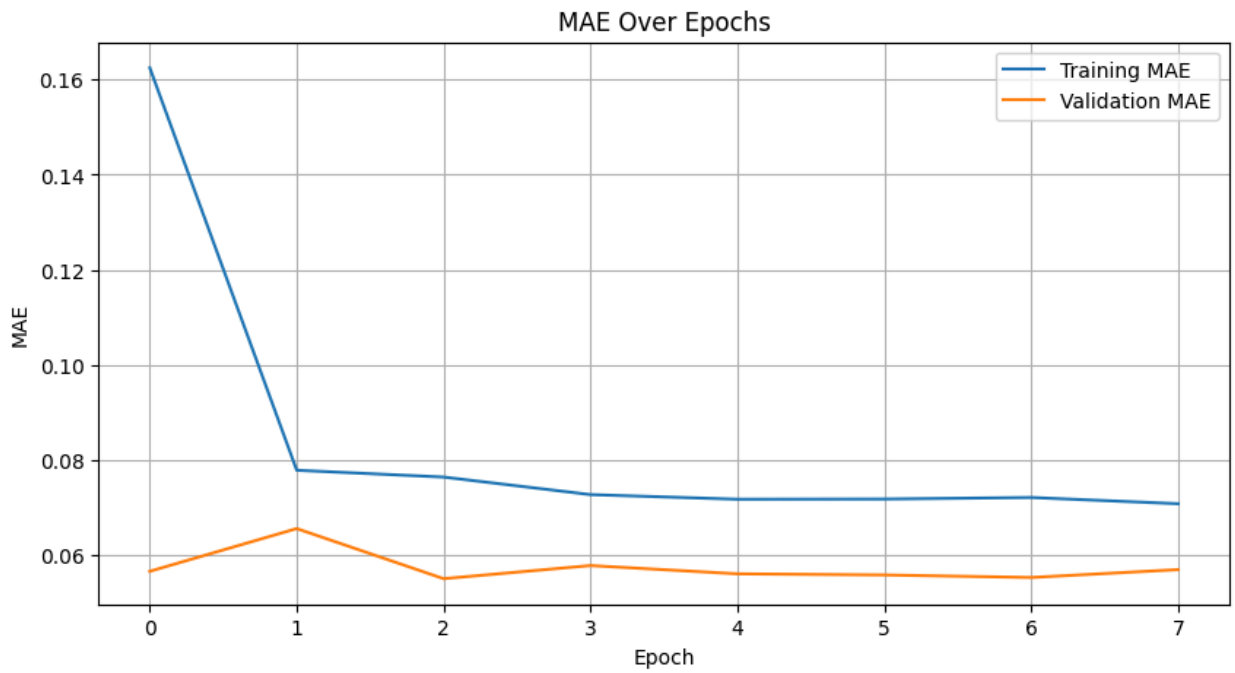


Figure 6.43 Training and validation MAE across epochs for the final LSTM–GRU model. Lower MAE indicates smaller average prediction error.

Return-to-Price Reconstruction

Because the model predicts returns rather than prices, an additional reconstruction step was required after inference. The predicted return was converted back into price using the previous known adjusted close value:

$$Predicted\ Price_{t+1} = Price_t \times (1 + Predicted\ Return_{t+1})$$

This allowed the model to preserve the stability advantages of return-based learning while still producing interpretable price forecasts for evaluation and visualization.

Predicting raw stock prices presents several challenges:

- Prices are **non-stationary** and trend over time
- Scale varies significantly across different stocks and time periods
- Small prediction errors can lead to large absolute deviations

To address this, the prediction target was transformed into **daily returns**, defined as:

$$Return_t = \frac{Price_t - Price_{t-1}}{Price_{t-1}}$$

<u>Significance of Phase 3</u>

Phase 3 represents the strongest and most complete version of the modelling process. More importantly, it demonstrates that model improvement in financial forecasting is not achieved by increasing complexity alone. The most important gains in this project came from:

- refining the prediction target
- simplifying the scope back to single-stock modelling
- introducing more informative engineered features
- using a hybrid recurrent design suited to sequential financial data

This phase therefore serves as the main contribution of the project and forms the basis for the final evaluation and forecasting results presented in the next section.

7. Evaluation Metrics and Future Forecasting

The final model was evaluated on a held-out test set consisting of the most recent 20% of observations. Because the Phase 3 model predicts next-day returns and then reconstructs them into price values, the evaluation needed to consider both **numerical forecasting accuracy** and **practical directional usefulness**. For this reason, the model was assessed using standard regression metrics, directional accuracy, and short-horizon future forecasting.

Stock market prediction is inherently difficult, and no model in this project is claimed to provide perfect forecasts or trading certainty. Instead, the purpose of this section is to show what the final model was able to achieve within the scope, time frame, and resources of the project.

7.1 Regression Evaluation Metrics

Since the final model reconstructs predicted returns into price values, standard regression metrics were used to compare predicted prices against actual prices on the unseen test set.

- **Mean Absolute Error (MAE)**: This metric represents the average absolute dollar difference between the model's predicted closing price and the actual asset price. A lower MAE immediately signifies a model highly capable of tracking a volatile asset's center of mass.
- **Root Mean Squared Error (RMSE)**: By squaring the errors prior to averaging, RMSE disproportionately penalizes large, catastrophic miscalculations. A minimal spread between the MAE and RMSE indicates that the model produces stable, consistent estimates without wildly overshooting trendlines.
- **R-Squared (R^2)**: This statistical metric confirms what percentage of the variance in the stock's actual movement is successfully captured and explained by the engineered feature set [16].

These metrics were useful for evaluating how well the return-based model could reconstruct stock prices. They also provided a direct comparison with the earlier raw-price forecasting approaches used in Phase 1 and Phase 2.

7.2 Directional Accuracy

Although regression metrics are important, they do not fully capture the practical usefulness of a stock prediction model. In financial forecasting, correctly identifying whether the price is likely to move **up or down** is often more actionable than minimizing average numerical error [20][27].

To evaluate this, directional accuracy was calculated by comparing the sign of consecutive predicted price changes with the sign of consecutive actual price changes. This measures how often the model correctly captured the next-day market direction.

```
pred_direction = (y_pred_prices[1:] > y_pred_prices[:-1]).astype(int)
actual_direction = (y_actual_prices[1:] > y_actual_prices[:-1]).astype(int)
```

Directional accuracy was especially important in this project because one of the main findings was that the final return-based model performed more strongly in capturing **market movement direction** than in producing exact price values [20][27]. This supports the practical idea that short-term trading decisions often depend more on direction than on precise price levels.

At the same time, this metric should still be interpreted carefully. A high directional score does not guarantee profitability, since real trading decisions also depend on transaction costs, timing, and market conditions that are outside the scope of this project.

7.3 Comparison between Phase 1, 2 and 3

The three phases of model development were designed to progressively improve the quality, stability, and practical usefulness of the stock prediction system.

Predicted vs Actual Price Chart

A time-series chart was produced comparing the model's predicted closing prices against the actual closing prices across the test period. The chart displays three series: the historical training data, the actual test prices, and the model's predictions.

7.31 Phase 1 Evaluation

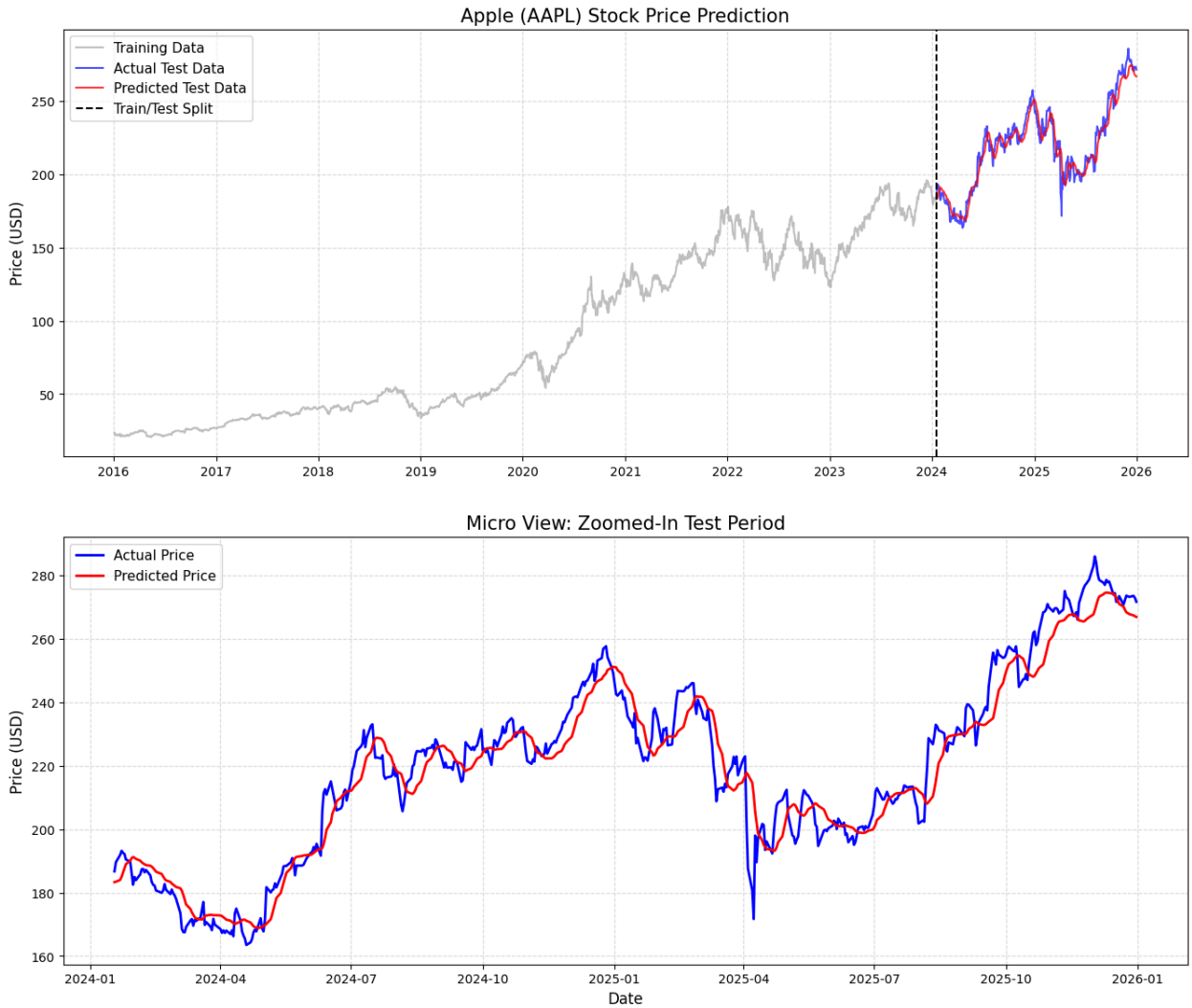


Figure 7.31 Phase 1 LSTM model for predicting AAPL stock prices using only the closing price as input. The top plot shows the full time series, including training data, actual test data, and predicted test values, with the vertical line indicating the 80/20 chronological train-test split. The bottom plot provides a zoomed-in view of the test period for clearer comparison.

The model is able to capture the overall trend of the stock price and produces predictions that generally follow the direction of the actual values. However, the predictions appear smoother and slightly lag behind sharp movements, indicating limited responsiveness to sudden market changes.

This behaviour reflects the simplicity of the model, which uses a single LSTM layer with 50 units and relies only on the closing price as input

Observations and Limitations (Phase 1)

Although the baseline model was able to learn general price trends, its overall performance was limited. More importantly, this phase highlighted a key challenge in stock forecasting: predicting **raw prices directly** is difficult because stock prices are non-stationary and strongly affected by market scale and volatility

As a result, predictions were often overly smooth and less responsive to sudden market changes.

7.32 Phase 2 Evaluation

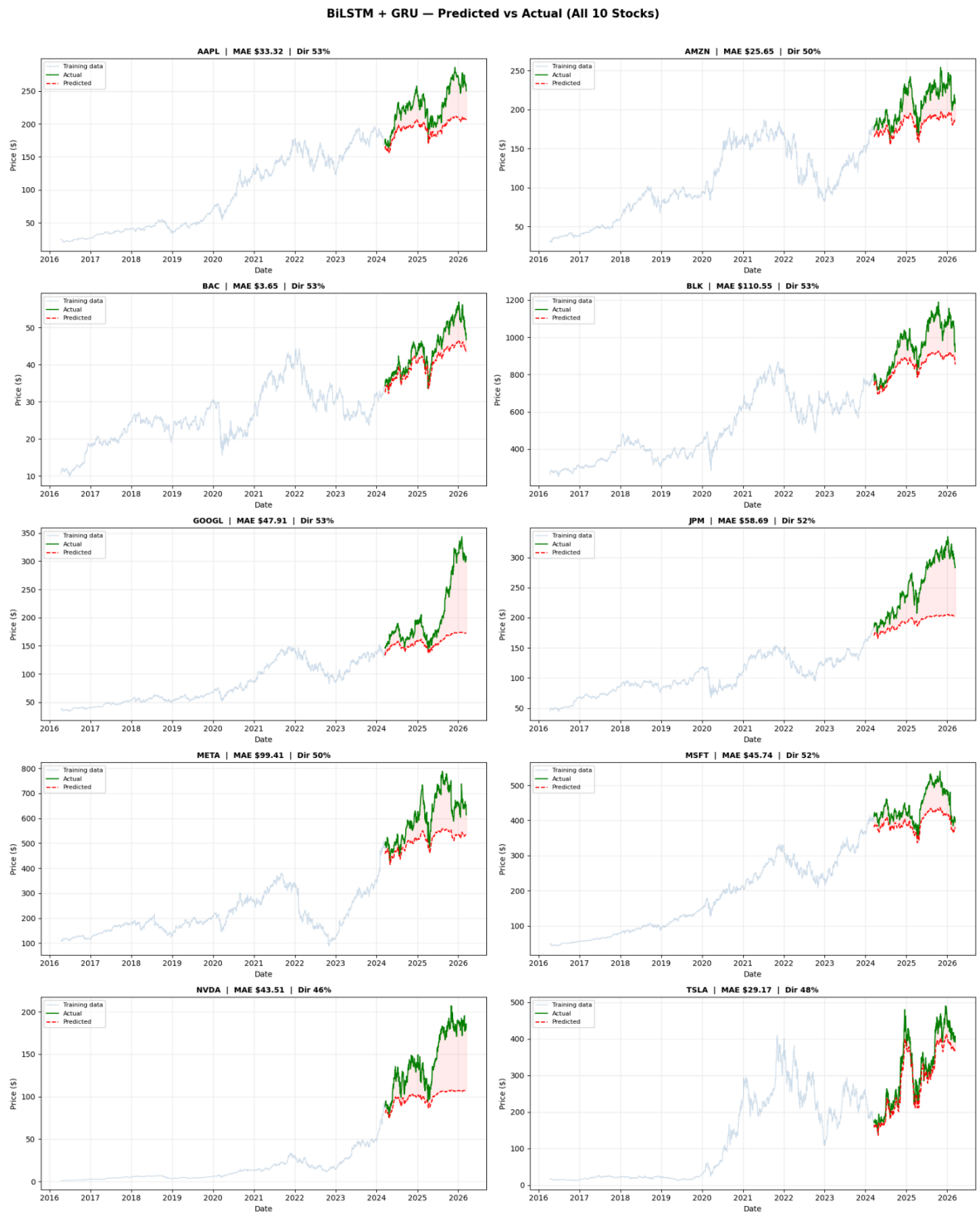
In Phase 2, incorporated a multistock model where 10 recommended stocks were chosen and past 10 years of data was fetched from Yahoo finance. The standard regression metrics were used, including **Mean Absolute Error (MAE)** and **Root Mean Squared Error (RMSE)**. These metrics were selected because the model in this phase still focused on predicting **next-day stock prices directly**, making it a regression-based forecasting problem.

```
==============================
  PER-STOCK PERFORMANCE REPORT
==============================

  AAPL                          BLK                           META                                          TSLA
    MAE              : $33.32     MAE              : $110.55    MAE              : $99.41                     MAE              : $29.17
    RMSE             : $37.18     RMSE             : $127.91    RMSE             : $114.50                    RMSE             : $34.24
    Price accuracy   : 85.98%     Price accuracy   : 89.14%     Price accuracy   : 84.71%                     Price accuracy   : 91.18%
    Direction acc    : 52.61%     Direction acc    : 53.01%     Direction acc    : 50.40%                     Direction acc    : 48.39%

  AMZN                          GOOGL                         MSFT
    MAE              : $25.65     MAE              : $47.91     MAE              : $45.74
    RMSE             : $28.25     RMSE             : $67.09     RMSE             : $50.89
    Price accuracy   : 88.06%     Price accuracy   : 80.24%     Price accuracy   : 90.01%
    Direction acc    : 49.80%     Direction acc    : 52.61%     Direction acc    : 52.01%

  BAC                           JPM                           NVDA
    MAE              : $3.65      MAE              : $58.69     MAE              : $43.51
    RMSE             : $4.38      RMSE             : $68.21     RMSE             : $50.57                     Average MAE  : $49.76
    Price accuracy   : 92.24%     Price accuracy   : 78.41%     Price accuracy   : 72.13%                     Average RMSE : $58.32
    Direction acc    : 53.01%     Direction acc    : 51.61%     Direction acc    : 45.78%                     Average Dir  : 50.9%
```

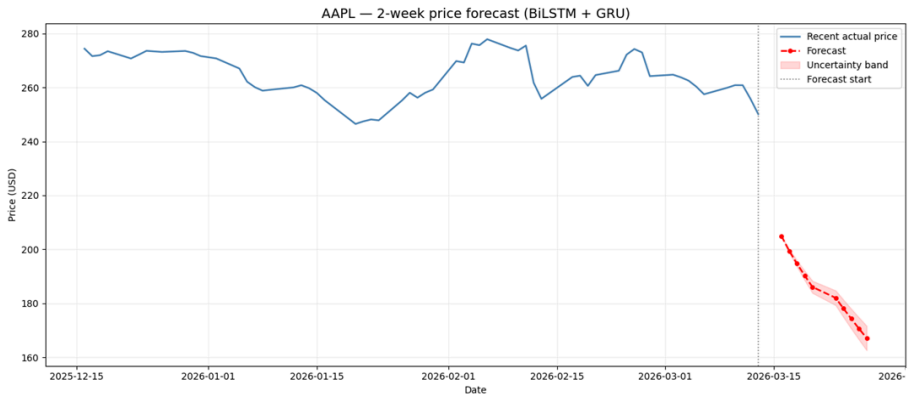
The results show that model **performance varies** significantly across different stocks. Some stocks, such as **TSLA and BAC**, achieve relatively lower error values and higher price accuracy, while others, such as **BLK and META**, exhibit higher prediction errors.

This variation highlights the challenge of using a single model to generalize across multiple stocks with different price ranges, volatility levels, and market behaviours.

On average, the model achieves an **MAE of approximately 49.76**, **RMSE of 58.32**, and **directional accuracy of around 50.9%**, indicating moderate regression performance but limited effectiveness in predicting price direction. The relatively low direction accuracy suggests that, although the model can approximate price values, it struggles to consistently capture short-term market movements.



Future Forecasting (14-Day Prediction)




```
2-WEEK FORECAST — AAPL
========================================
Last close (2026-03-13) : $250.12
----------------------------------------
Date            Predicted      Change
----------------------------------------
2026-03-16     $  204.93  ▼ $ 45.19
2026-03-17     $  199.43  ▼ $  5.51
2026-03-18     $  194.71  ▼ $  4.71
2026-03-19     $  190.28  ▼ $  4.44
2026-03-20     $  186.07  ▼ $  4.21
2026-03-23     $  182.01  ▼ $  4.05
2026-03-24     $  178.10  ▼ $  3.91
2026-03-25     $  174.33  ▼ $  3.78
2026-03-26     $  170.66  ▼ $  3.67
2026-03-27     $  167.11  ▼ $  3.55
----------------------------------------
2-week outlook : DOWN  $83.01 total
========================================
```

Figure 7.321 shows the 2-week price forecast for AAPL using the BiLSTM + GRU model. The forecast demonstrates a strong downward bias, where the model consistently predicts a declining trend regardless of recent price behaviour. This indicates poor generalization in future forecasting.

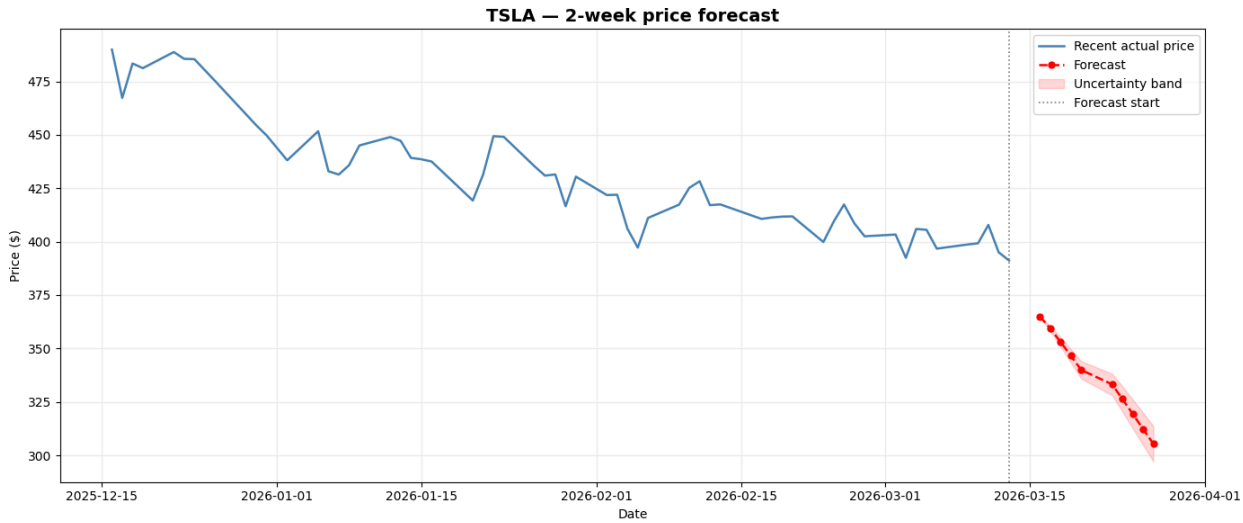


Figure 7.322 shows the 2-week forecast for TSLA, where a similar pattern is observed. The model again predicts a continuous downward movement, suggesting that it fails to adapt to dynamic market trends and instead exhibits a biased prediction behaviour.

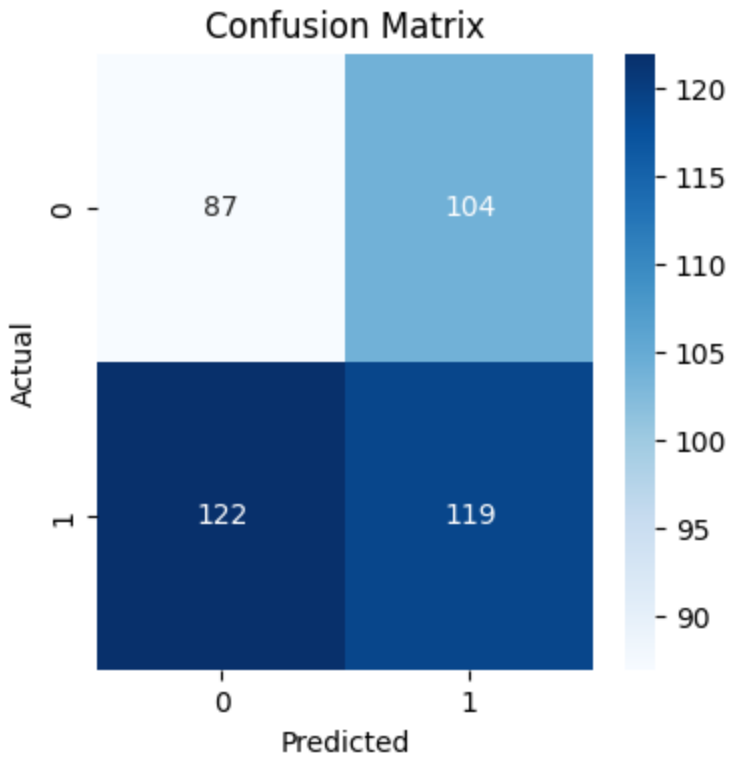


Figure 7.323 presents the confusion matrix for the direction classification task. The results show a high number of misclassifications in both classes (upward and downward movement), indicating that the model struggles to correctly predict market direction.

```
=====================================
  BASELINE — ALWAYS PREDICT UP
=====================================
AAPL: 55.79%
AMZN: 51.47%
BAC: 54.09%
BLK: 52.80%
GOOGL: 56.45%
JPM: 57.88%
META: 51.92%
MSFT: 53.57%
NVDA: 55.15%
TSLA: 50.22%

Baseline Average Accuracy: 53.93%
```

Observations and Limitations (Phase 2)

Phase 2 was evaluated to determine whether a more advanced hybrid recurrent architecture could improve forecasting performance beyond the baseline LSTM model developed in Phase 1.

However, despite these improvements, there are several limitations. **First**, the model still relied on **direct price prediction**, which remained sensitive to non-stationarity, scale differences, and volatility. **Second**, the **multi-stock setup** introduced inconsistency across assets, since different stocks exhibited different behavioural characteristics and volatility patterns. This reduced the model's ability to generalise uniformly across all securities.

Moreover, some of the supporting direction-based experiments conducted during this phase showed that the model did not consistently outperform simple baselines in all cases.

For this reason, Phase 2 should be viewed as an experiment but incomplete improvement. It clearly represents the complex architecture from the baseline Phase 1 model in numerical forecasting details, but it also highlighted the need for a better-formulated prediction target and a more focused modelling scope.

Overall, this phase was critical to the project journey. It provided evidence that hybrid architectures could improve performance, but it also showed that the most important remaining challenge was not only architectural complexity, but also the formulation of the forecasting problem itself.

7.33 Phase 3 Evaluation

Phase 3 represents the **final and most effective stage** of the project. In this phase, the modelling approach was reformulated to predict **next-day returns instead of raw prices**, and these returns were later reconstructed into actual price values. The motivation and structure of this phase were already established in the model development section.

This change, along with:

- a **single-stock focus (TSLA)**
- and **engineered input features**

resulted in significantly more **stable, accurate, and interpretable predictions** compared to previous phases.

Regression Performance

When evaluated on the held-out test set, the final model achieved:

- **MAE:** 9.3594
- **RMSE:** 12.3902
- **R^2 Score:** 0.9785

These results indicate **improved predictive performance**, with predictions closely matching actual stock prices.

Actual vs Predicted Prices

The model closely follows the **true TSLA price trend over time.**

Both **macro (full timeline)** and **micro (zoomed test view)** confirm strong alignment

- Minor deviations occur during **high volatility periods**, which is expected in financial data

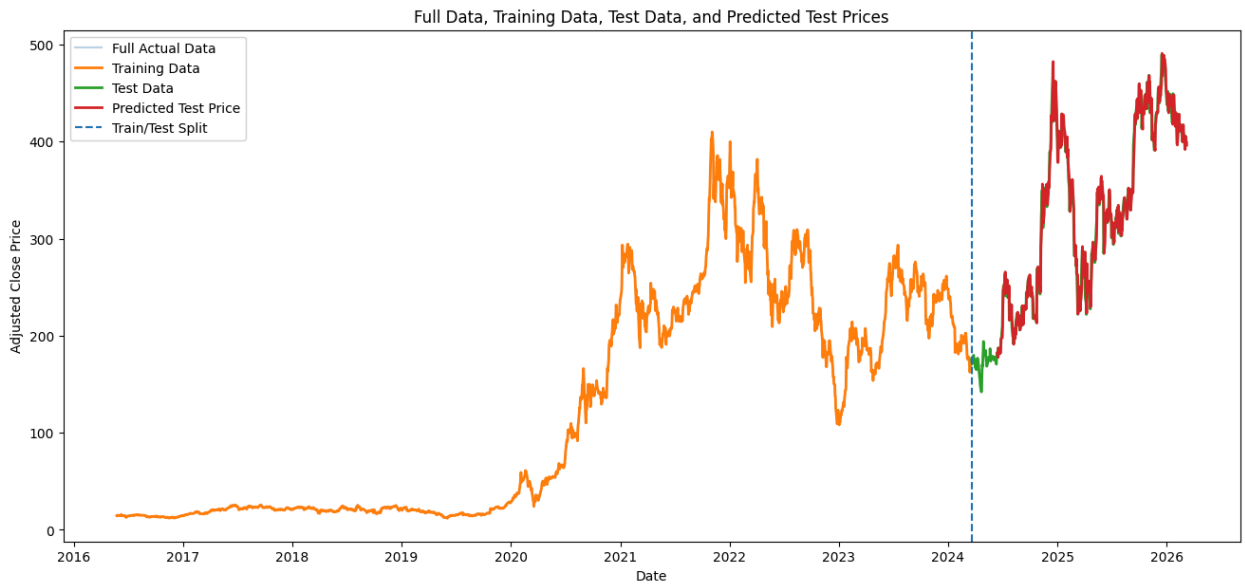


Figure 7.331

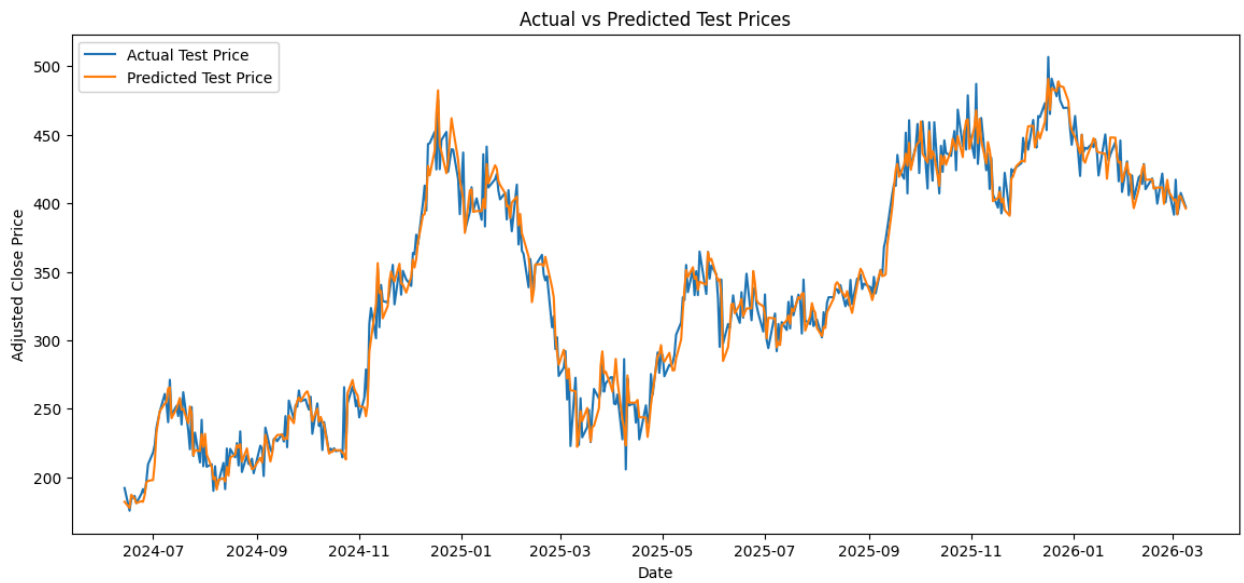


Figure 7.332

Figure 7.331 & 7.332 demonstrates that the model captures **overall trend and short-term movements effectively**

Scatter Plot: Actual vs Predicted Prices

Data points lie closely along the **diagonal (ideal fit line)**

Confirms the **high R^2 value visually**

Indicates strong correlation between **actual and predicted prices**

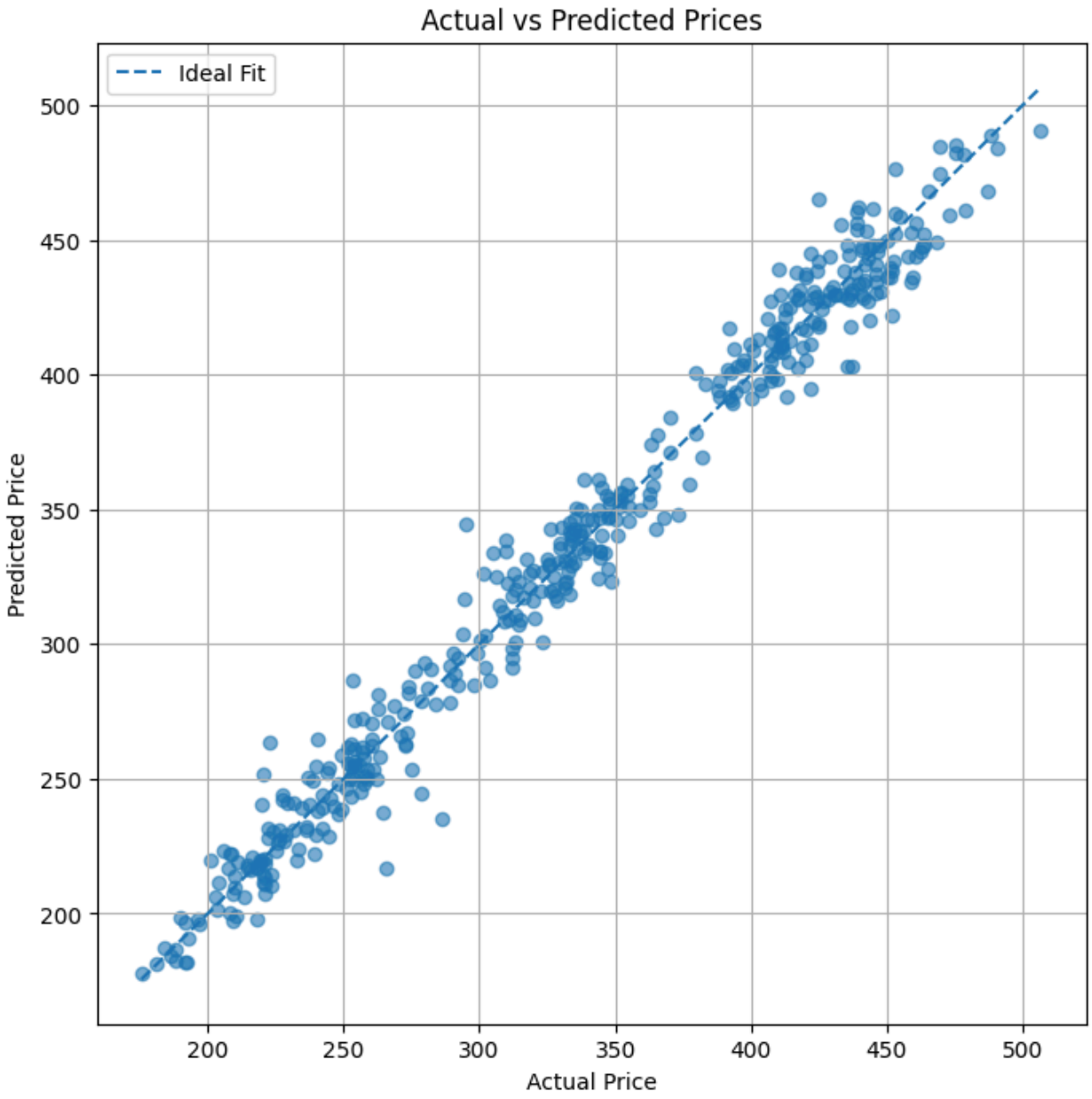


Figure 7.333 Shows the model is numerically accurate and well-calibrated for Phase 3

Residual Analysis

Errors fluctuate around **zero**
No clear systematic pattern observed

Indicates:

- No strong bias (under/over prediction)
- Model is **well-balanced**

Error Distribution

- Errors are **normally distributed around zero**
- Most predictions fall within a **small error range**

Suggests **high prediction stability** and **reduced extreme errors** compared to earlier phases.

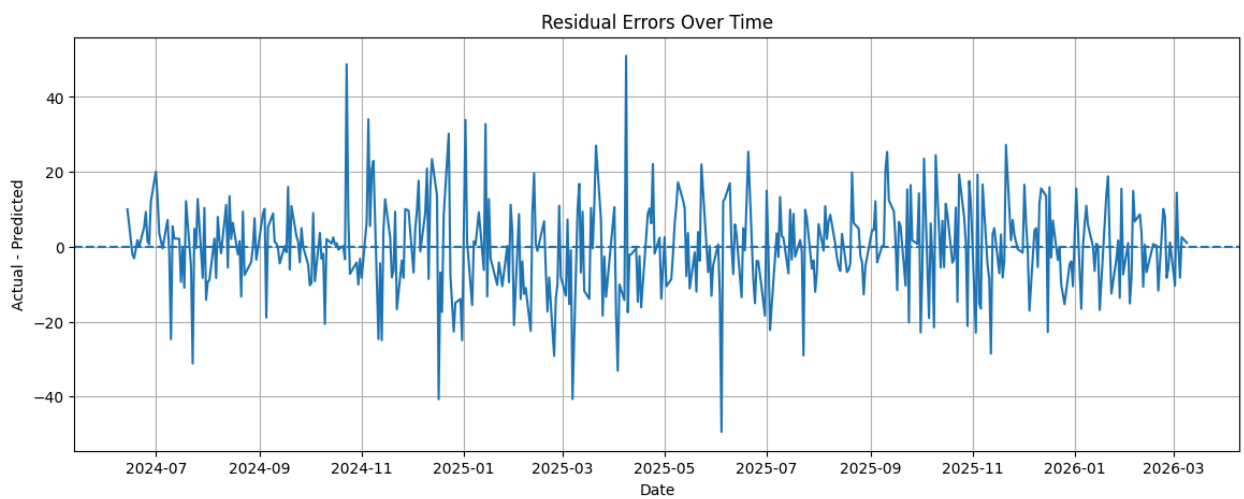


Figure 7.334 Phase 3, residual errors over the test period, showing where the model overestimated or underestimated stock prices.

Direction Prediction Performance

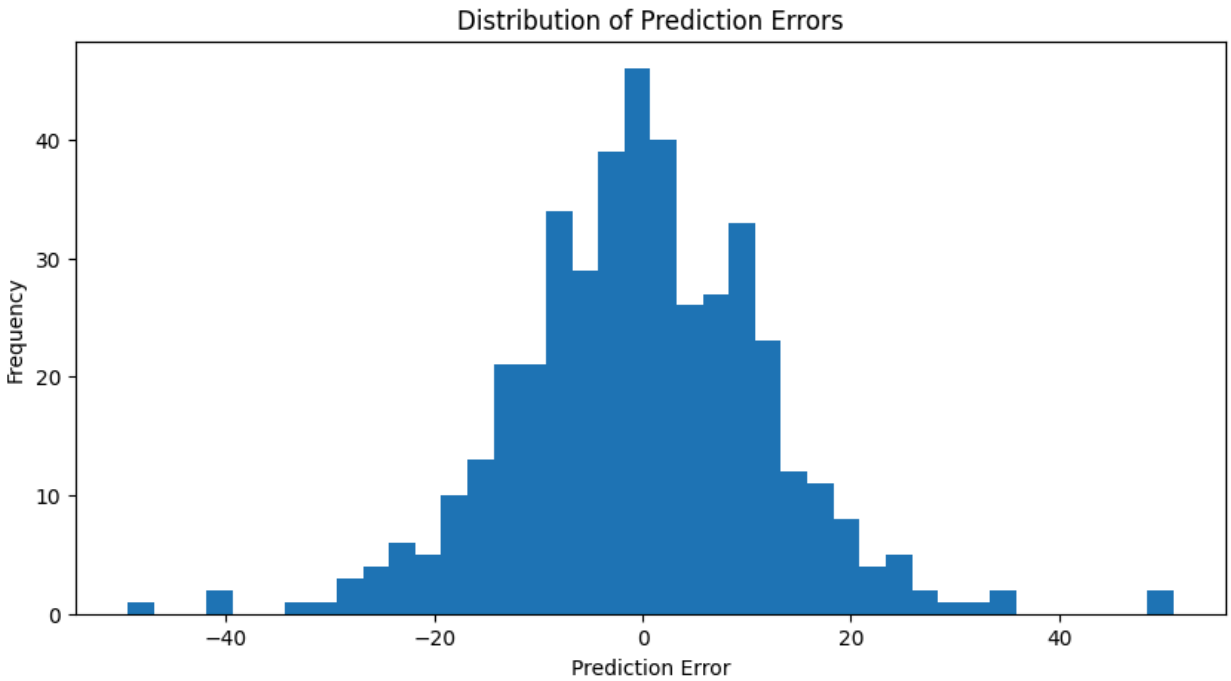


Figure 7.335 Distribution of prediction errors for the final model. A tighter concentration around zero indicates more stable predictions.

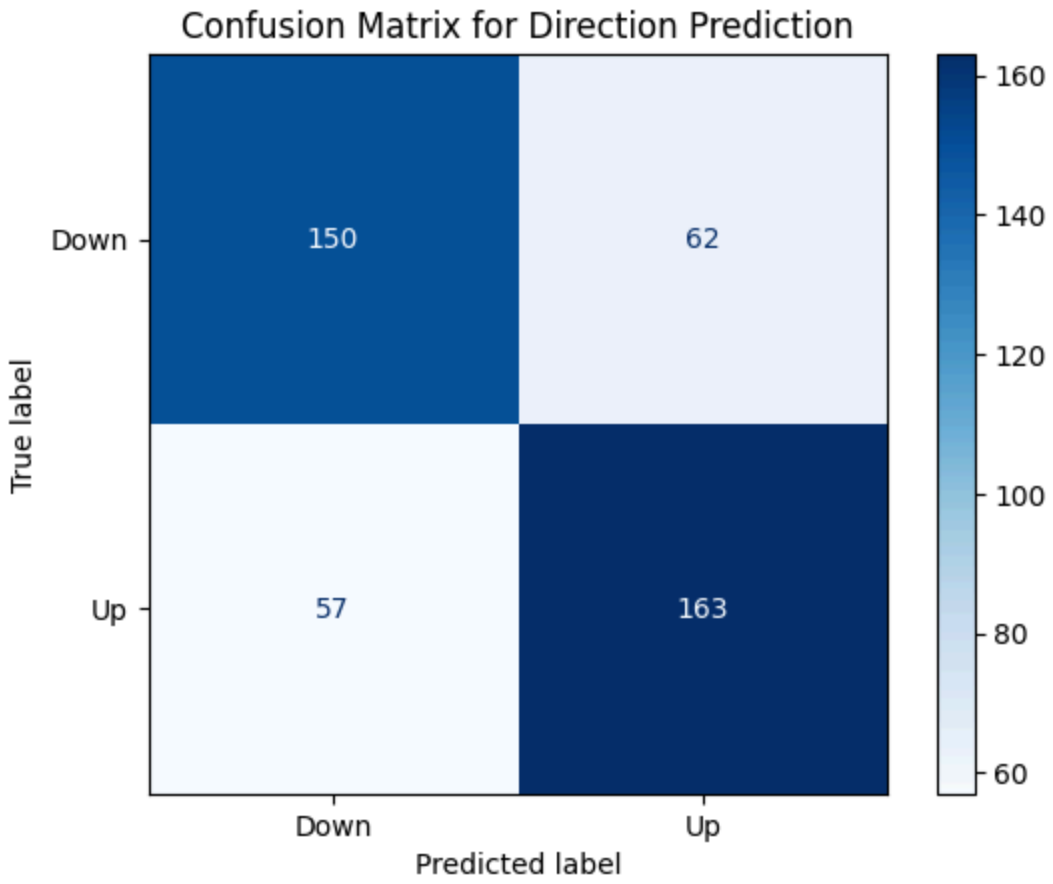
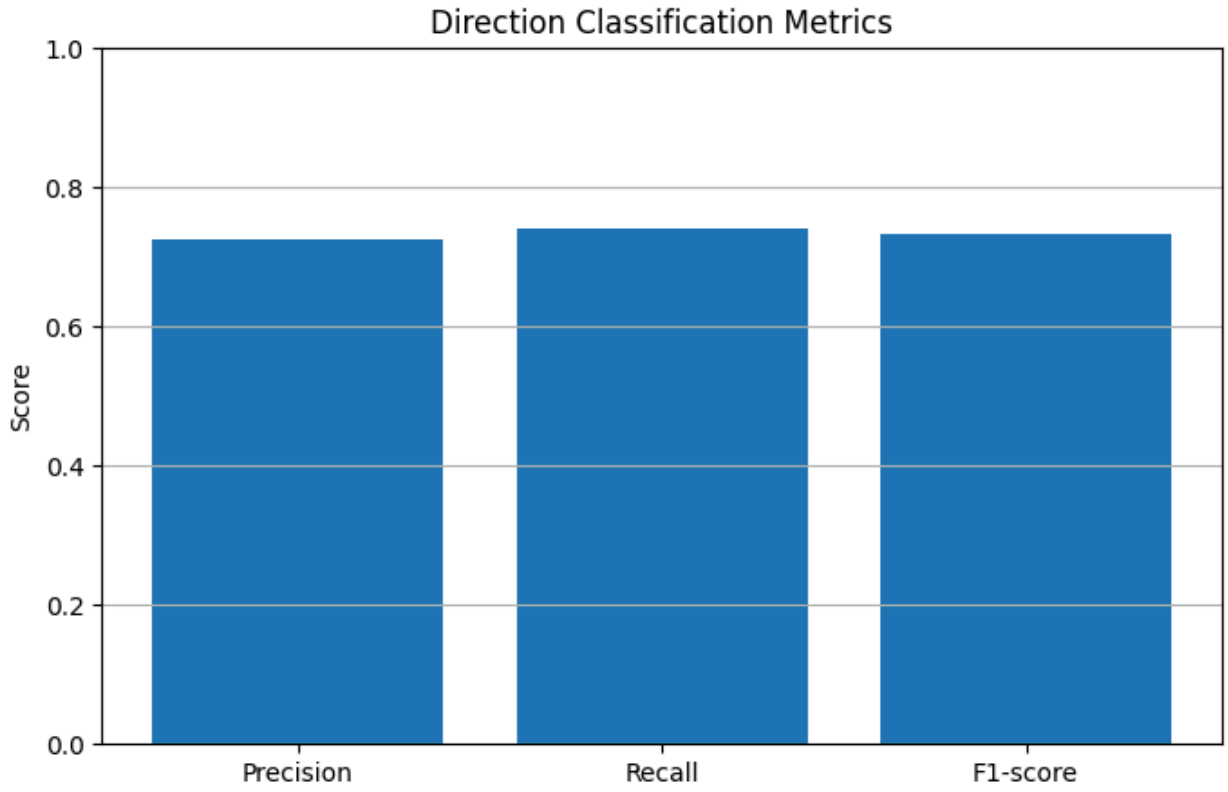


Figure 7.336 Confusion matrix showing the ability of the final model to correctly predict upward and downward price movement based on reconstructed prices.

Classification Metric



- **Precision ≈ 0.73**
- **Recall ≈ 0.74**
- **F1-score ≈ 0.73**

This is a **major improvement over Phase 2 (~50%)**, showing strong directional learning.

Future Forecasting (14-Day Prediction)

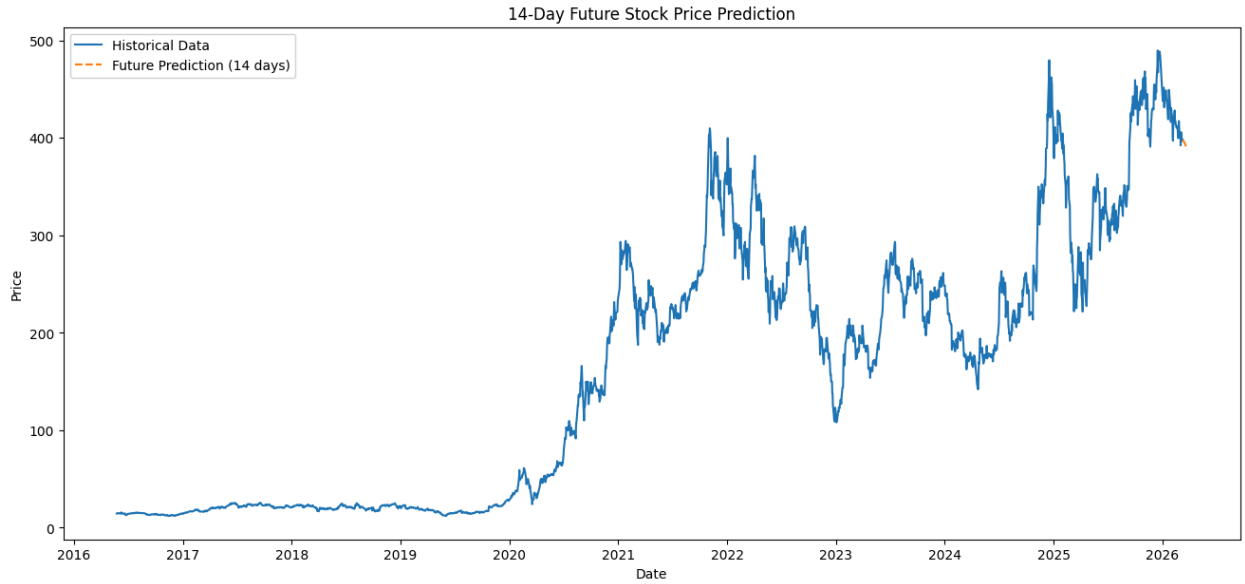


Figure 7.338 Tesla Stock 14 day future Price Prediction with Overview of the Dataset

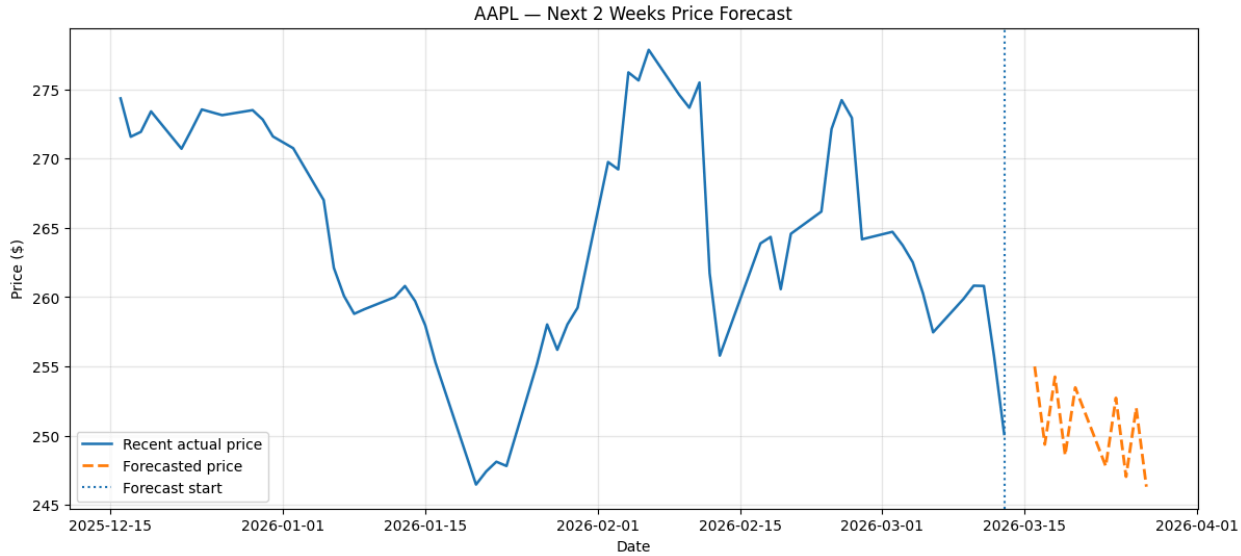


Figure 7.339 Apple Stock 14 day future Price Prediction (Same model with Bidirection LSTM)

Observation and Limitations

The high **R^2 score (0.9785)** indicates that the model successfully captured a large portion of the variation in TSLA's price behaviour during the test period. However, this should not be

interpreted as perfect forecasting accuracy. Financial markets remain inherently volatile and are influenced by external factors such as macroeconomic events, earnings reports, and geopolitical developments, which are not included in the model inputs

Model Fit and Behaviour

- The **scatter plot (Actual vs Predicted)** shows that most points lie close to the ideal-fit line
- Indicates strong alignment between predicted and actual prices
- Some deviation occurs during **high-volatility periods**

Comparison with Naive Baseline

A naive baseline was also used for comparison, where the next day's price was assumed to remain unchanged from the previous day. This produced:

- **Baseline MAE:** 9.3096
- **Baseline RMSE:** 12.3820

Compared to the model:

- **Model MAE:** 9.36
- **Model RMSE:** 12.39

This comparison is important because it shows that precise short-term stock forecasting is extremely difficult. Although the final model performed strongly in terms of overall structure and fit, its regression error was only slightly different from a simple baseline. This is not a failure of the project; rather, it reflects a well-known property of financial time-series data, where naive forecasts can remain highly competitive over short horizons because consecutive daily prices are naturally autocorrelated.

For this reason, the strongest contribution of the final model was not in dramatically outperforming the naive baseline on raw price error alone, but in providing a more meaningful **directional interpretation** of market behaviour.

Directional Performance

To evaluate whether the model was able to identify short-term market movement, directional predictions were derived by comparing consecutive reconstructed predicted prices with consecutive actual prices.

The final model achieved:

- **Directional Accuracy:** 0.7245
- **Precision:** 0.7244
- **Recall:** 0.7409
- **F1-score:** 0.7326

In addition, the predicted **class balance** remained close to the actual balance:

- **Predicted UP %:** 52.08%
- **Actual UP %:** 50.93%

These results indicate that the final model was not simply overpredicting one class, but was producing relatively balanced directional forecasts. The confusion matrix also showed a good spread of correct upward and downward classifications, further supporting the practical usefulness of the model in identifying short-term directional movement.

This directional performance was substantially stronger than simple baseline strategies. The notebook outputs showed:

- Always-UP baseline accuracy: 0.5093
- Majority-class baseline accuracy: 0.5093
- Random baseline accuracy: approximately 0.52

Compared with these values, the final model's directional accuracy of approximately 72.45% represents a meaningful improvement. This was one of the strong findings of the project and supports the conclusion that the final return-based formulation was more useful for identifying market direction than the earlier raw-price approaches.

Actual vs Predicted Price Behaviour

The predicted-versus-actual time-series graph showed that the final model was able to follow the broader direction of the TSLA price series while smoothing some of the short-term fluctuations. This behaviour is expected in a return-based recurrent forecasting model. The model captured major movements and overall trend shifts more effectively than the baseline phases, but it was still less precise during abrupt changes and highly volatile intervals.

The residual plots and error histogram also supported this interpretation. Errors were generally centered around zero, indicating no severe systematic bias, but some larger deviations remained during more difficult regions of the test set. This again reflects the fact that stock market prediction is inherently uncertain and that even improved models cannot fully capture sudden market discontinuities.

Next-Day Forecast and Short Future Horizon

For a one-step-ahead forecast, the final model predicted:

- **Last known price:** 398.68
- **Predicted return:** -0.0053661
- **Predicted next-day price:** 398.17
- **Predicted direction:** Down

This output is useful because it shows how the model can produce both a numerical forecast and an interpretable directional signal.

A short 14-day recursive forecast was also generated. The future predictions gradually declined from approximately **398.17** to **388.01** over the projected period. However, this forecast should be interpreted cautiously. Recursive forecasting compounds uncertainty because each future prediction depends on previously predicted values rather than fully observed market data. In addition, some engineered features must be approximated during recursive forecasting, which further reduces reliability over longer horizons.

For this reason, the future forecast in Phase 3 should be interpreted primarily as a **short-term directional outlook** rather than a precise price path. It is most credible over the earliest forecast steps and becomes less certain as the forecast horizon extends.

Overall Interpretation of Phase 3

Phase 3 provided the most balanced outcome of the project by combining strong directional forecasting with stable return-based regression, while still acknowledging the inherent uncertainty and unpredictability of financial markets. Stock prediction remains fundamentally difficult, and short-term prices are highly sensitive to unpredictable external events. However, within the scope of this project, the final return-based LSTM–GRU model clearly represented the strongest and most meaningful outcome.

Its main strengths were:

- improved training stability
- strong fit to the general price structure
- balanced and accurate directional forecasting
- better alignment between problem formulation and financial data behaviour

Most importantly, Phase 3 demonstrated that the project's most valuable improvement came not only from changing the model architecture, but from reformulating the task itself. By moving from raw-price prediction to return-based forecasting, the final model became more stable, more interpretable, and more practically useful.

8. Project Reflection

A. Key Project Successes (What Worked Well)

This project successfully developed a complete **end-to-end stock prediction pipeline**, covering all major stages of a machine learning workflow. These stages included data collection, preprocessing, feature engineering, model training, evaluation, and short-horizon forecasting.

A major strength of the project was its structured progression across three development phases:

- **Phase 1:** A baseline LSTM model using raw price data
- **Phase 2:** A more advanced BiLSTM + GRU hybrid model applied to multiple stocks
- **Phase 3:** A refined return-based LSTM–GRU model focused on a single stock

One of the most important improvements was the **reformulation of the prediction problem**. Instead of predicting raw prices, the final model predicted **returns**, which significantly reduced scale sensitivity and improved model stability and interpretability.

The project also developed a meaningful **engineered feature space**, including:

- Adjusted Close
- Returns
- Moving Averages (MA10, MA20, MA50)
- Volatility (Volatility10)
- Volume Change

Additionally, proper **time-series methodology** was followed throughout:

- Chronological train-test split
- Scaling applied only on training data
- Strict prevention of data leakage

The final Phase 3 model achieved the strongest performance, demonstrating:

- Stable training behaviour
- Strong alignment with actual price trends
- Significantly improved directional accuracy compared to naive baselines

These successes were supported by a range of effective visualizations, including:

- Chronological train-test split representation
- Feature scaling comparisons

- Feature engineering visualizations (multi-panel plots)
- Training loss and MAE curves
- Actual vs predicted price comparisons
- Confusion matrix and direction performance charts

B. Technical Challenges Encountered

Several technical challenges were encountered throughout the project.

One of the primary difficulties was **direct price prediction**. Raw stock prices are inherently non-stationary and scale-dependent, which made them difficult for models to learn effectively. As a result, predictions often appeared overly smooth and failed to capture short-term fluctuations.

The **multi-stock modelling approach** in Phase 2 also introduced challenges. Different stocks exhibited varying volatility patterns and behaviours, making it difficult for a single model to generalize consistently across all assets.

Another critical challenge was **preventing data leakage**. Since the data is time-series in nature, it could not be randomly split. Instead, a chronological split was required, and feature scaling had to be performed using only training data to ensure realistic evaluation.

The project also encountered limitations in **recursive forecasting**. In multi-step predictions, each future value depends on previously predicted values, causing errors to accumulate over time and reducing reliability for longer forecasting horizons.

Additionally, **naive baseline models remained competitive**, particularly for short-term price prediction. Due to the natural autocorrelation in stock prices, simple “no-change” models often performed similarly to more complex models when evaluated using raw price error metrics.

Finally, the **inherent unpredictability of financial markets** remained a fundamental limitation. External factors such as geopolitical events, earnings announcements, macroeconomic conditions, and investor sentiment significantly influence market behaviour but were not included in the model.

C. Technical Skills Gained and Lessons Learned

This project significantly improved understanding of **time-series forecasting concepts**, including sequential data preparation, lookback window design, and chronological validation strategies.

Practical experience was gained in implementing and working with advanced deep learning architectures, including:

- LSTM
- Bidirectional LSTM (BiLSTM)
- GRU
- Hybrid recurrent models

The project also reinforced the importance of **feature engineering**, demonstrating that raw data alone is often insufficient and that well-designed input features can substantially improve model performance.

One of the most important lessons learned was that **problem formulation is as critical as model design**. The most significant improvement in performance came not from increasing model complexity, but from shifting the prediction target from prices to returns.

Additionally, strong skills were developed in **model evaluation**, including the use and interpretation of:

- MAE, RMSE, and R^2
- Directional accuracy
- Precision, Recall, and F1-score
- Baseline comparisons

Finally, the project emphasized the importance of **honest and realistic evaluation**. A good statistical fit does not guarantee perfect prediction, particularly in financial markets. In many practical scenarios, correctly predicting market direction is more valuable than achieving minimal numerical error.

9. Conclusion and Future Work

This project explored the development of a stock market prediction system through a structured three-phase modelling process. The work began with a simple LSTM baseline using raw closing prices, progressed to a more advanced BiLSTM–GRU hybrid model in a multi-stock setting, and ultimately concluded with a final **return-based LSTM–GRU model** trained in a focused single-stock setting. This progression was one of the main strengths of the project, as it demonstrated continuous improvement through experimentation, comparison, and critical evaluation rather than reliance on a single approach.

A key conclusion of the project is that **problem formulation played a more important role than architectural complexity alone**. While the move from Phase 1 to Phase 2 showed that richer recurrent architectures could reduce error and improve feature extraction, the most meaningful improvement came in Phase 3 when the prediction target changed from **raw price** to **next-day return**. This reformulation reduced scale sensitivity, improved stability, and made the model more suitable for financial time-series forecasting. The final return-based LSTM–GRU model therefore represented the most balanced and practical solution developed in this study.

The project also showed that stock market prediction remains inherently difficult [1]. Even with feature engineering, proper preprocessing, hybrid recurrent architectures, and careful evaluation, the model could not perfectly forecast price behaviour [32]. This is consistent with the realities of financial markets, where prices are influenced not only by historical trends, but also by external events such as earnings announcements, policy changes, geopolitical conflict, recessions, and sudden market sentiment shifts. For this reason, the final results should be interpreted as meaningful but limited [29][33]. The model was strongest in capturing **general price structure** and especially **directional movement**, rather than producing exact price forecasts with complete reliability.

Another important conclusion is that **honest evaluation matters**. The project showed that naive baselines remain competitive in raw price forecasting because of the autocorrelated nature of stock prices [26]. However, the final model provided stronger practical value through improved directional forecasting and a more informative, return-based interpretation of market movement. This made the project more realistic and academically stronger, since the report does not claim success but instead explains both the strengths and limitations of the final system.

<u>Future Work</u>

Several meaningful improvements can be made in future work to extend the value of this project:

- **Integrate an AI-based external event analysis module**
  A future version of this system could combine the current machine learning model with

an AI model that analyzes external events such as recessions, wars, inflation shocks, policy changes, and pandemics such as COVID-19. The current model relies mainly on historical price and volume data, which means it cannot directly capture the impact of geopolitical and macroeconomic developments. Adding an AI-based event or sentiment analysis component could help the forecasting model make more context-aware predictions.

- **Incorporate financial news and sentiment data**
  News headlines, earnings reports, analyst sentiment, and social media signals could be added as supplementary inputs to improve responsiveness to real-world market behaviour [19].
- **Improve multi-step forecasting**
  The current recursive future forecasting method accumulates error over time. A future model trained directly for multi-step forecasting could improve reliability over longer horizons.
- **Explore stronger multi-stock generalization methods**
  Future work could revisit the multi-stock setting using sector-specific models, stock embeddings, or transfer learning techniques to better preserve stock-specific behaviour.
- **Add uncertainty estimation**
  Instead of outputting only one predicted value, future systems could provide confidence intervals or forecast ranges, giving a more realistic view of uncertainty in stock prediction.
- **Compare with additional forecasting models**
  Future research could benchmark the final model against ARIMA, XGBoost, Transformer-based time-series models, and other modern forecasting approaches [29].

References

[1]
E. Fama, “Efficient Capital markets: a Review of Theory and Empirical Work,” *The Journal of Finance*, vol. 25, no. 2, pp. 383–417, May 1970, doi: https://doi.org/10.2307/2325486.
[2]
G. T. Wilson, “Time Series Analysis: Forecasting and Control, 5th Edition,” *Journal of Time Series Analysis*, vol. 37, no. 5, pp. 709–711, Mar. 2016, doi: https://doi.org/10.1111/jtsa.12194.
[3]
R. J. Hyndman and G. Athanasopoulos, *Forecasting : Principles and Practice*, 2nd ed. Heathmont, Vic.: Otexts, 2018. Available: https://otexts.com/fpp2/
[4]
A. A. Ariyo, A. O. Adewumi, and C. K. Ayo, “Stock Price Prediction Using the ARIMA Model,” *2014 UKSim-AMSS 16th International Conference on Computer Modelling and Simulation*, Mar. 2014, doi: https://doi.org/10.1109/uksim.2014.67.
[5]
G. S. Atsalakis and K. P. Valavanis, “Surveying stock market forecasting techniques – Part II: Soft computing methods,” *Expert Systems with Applications*, vol. 36, no. 3, pp. 5932–5941, Apr. 2009, doi: https://doi.org/10.1016/j.eswa.2008.07.006.
[6]
J. Patel, S. Shah, P. Thakkar, and K. Kotecha, “Predicting stock market index using fusion of machine learning techniques,” *Expert Systems with Applications*, vol. 42, no. 4, pp. 2162–2172, Mar. 2015, doi: https://doi.org/10.1016/j.eswa.2014.10.031.
[7]
W.-C. Lin, C.-F. Tsai, and H. Chen, “Factors affecting text mining based stock prediction: Text feature representations, machine learning models, and news platforms,” *Applied Soft Computing*, vol. 130, no. 1568–4946, p. 109673, Nov. 2022, doi: https://doi.org/10.1016/j.asoc.2022.109673.
[8]
Y. LeCun, Y. Bengio, and G. Hinton, “Deep Learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015, doi: https://doi.org/10.1038/nature14539.
[9]
T. Fischer and C. Krauss, “Deep learning with long short-term memory networks for financial market predictions,” *European Journal of Operational Research*, vol. 270, no. 2, pp. 654–669, Oct. 2018, doi: https://doi.org/10.1016/j.ejor.2017.11.054.
[10]
A. Moghar and M. Hamiche, “Stock Market Prediction Using LSTM Recurrent Neural Network,” *Procedia Computer Science*, vol. 170, pp. 1168–1173, 2020, doi: https://doi.org/10.1016/j.procs.2020.03.049.
[11]
H. Wen, “Prediction of Stock Price by Neural Network Based on CNN, LSTM, ANN,” *Advances in Economics, Management and Political Sciences*, vol. 87, no. 1, pp. 229–237, Jun. 2024, doi: https://doi.org/10.54254/2754-1169/87/20240899.
[12]

A. Farhadi, A. Zamanifar, A. Alipour, A. Taheri, and M. Asadolahi, “A Hybrid LSTM-GRU Model for Stock Price Prediction,” *IEEE Access*, vol. 13, no. 2169–3536, pp. 117594–117618, 2025, doi: https://doi.org/10.1109/access.2025.3586558.
[13]
H. Abdelsalam, A. Souri, and N. İnanç, “A time series forecasting approach based on gradient boosting method for IoT-based solar energy production systems,” *Energy Storage and Saving*, no. S2772683525000366, Aug. 2025, doi: https://doi.org/10.1016/j.enss.2025.04.003.
[14]
S. Khan *et al.*, “Deep learning model for efficient traffic forecasting in intelligent transportation systems,” *Neural Computing and Applications*, no. 0941–0643, Dec. 2024, doi: https://doi.org/10.1007/s00521-024-10537-z.
[15]
D. B *et al.*, “AI-Based Energy Consumption Forecasting in Smart Homes Using Multi-Algorithm Analysis Across Temporal Resolutions,” *Results in Engineering*, no. 2590–1230, p. 109272, Feb. 2026, doi: https://doi.org/10.1016/j.rineng.2026.109272.
[16]
T. Chai and R. R. Draxler, “Root mean square error (RMSE) or mean absolute error (MAE)? – Arguments against avoiding RMSE in the literature,” *Geoscientific Model Development*, vol. 7, no. 3, pp. 1247–1250, Jun. 2014, doi: https://doi.org/10.5194/gmd-7-1247-2014.
[17]
C. Bergmeir, R. J. Hyndman, and B. Koo, “A note on the validity of cross-validation for evaluating autoregressive time series prediction,” *Computational Statistics & Data Analysis*, vol. 120, no. 0167-9473, pp. 70–83, Apr. 2018, doi: https://doi.org/10.1016/j.csda.2017.11.003.
[18]
M. M. Kumbure, C. Lohrmann, P. Luukka, and J. Porras, “Machine learning techniques and data for stock market forecasting: A literature review,” *Expert Systems with Applications*, vol. 197, no. 1, p. 116659, Jul. 2022, doi: https://doi.org/10.1016/j.eswa.2022.116659.
[19]
A. Atkins, M. Niranjan, and E. Gerding, “Financial news predicts stock market volatility better than close price,” *The Journal of Finance and Data Science*, vol. 4, no. 2, pp. 120–137, Jun. 2018, doi: https://doi.org/10.1016/j.jfds.2018.02.002.
[20]
M. Qiu and Y. Song, “Predicting the Direction of Stock Market Index Movement Using an Optimized Artificial Neural Network Model,” *PLOS ONE*, vol. 11, no. 5, p. e0155133, May 2016, doi: https://doi.org/10.1371/journal.pone.0155133.
[21]
O. B. Sezer, M. U. Gudelek, and A. M. Ozbayoglu, “Financial time series forecasting with deep learning : A systematic literature review: 2005–2019,” *Applied Soft Computing*, vol. 90, p. 106181, May 2020, doi: https://doi.org/10.1016/j.asoc.2020.106181.
[22]
B. Lim and S. Zohren, “Time Series Forecasting With Deep Learning: A Survey,” *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 379, no. 2194, p. 20200209, Apr. 2021, doi: https://doi.org/10.1098/rsta.2020.0209..

[23]
Y. Zhao, “Stock Prediction based on LSTM Neural Network,” *Highlights in Business, Economics and Management*, vol. 3, pp. 19–23, Jan. 2023, doi: https://doi.org/10.54097/hbem.v3i.4630.
[24]
A. Dutta, Gopu Pooja, N. Jain, R. R. Panda, and Naresh Kumar Nagwani, “A Hybrid Deep Learning Approach for Stock Price Prediction,” *Lecture notes in networks and systems*, pp. 1–10, Oct. 2020, doi: https://doi.org/10.1007/978-981-15-7106-0_1.
[25]
X. Jin, C. Dai, Y. Tao, and F. Zhang, “A comparative study of RNN, CNN, LSTM and GRU for random sea wave forecasting,” *Ocean Engineering*, vol. 338, pp. 122013–122013, Jun. 2025, doi: https://doi.org/10.1016/j.oceaneng.2025.122013.
[26]
R. S. Tsay, *Analysis of Financial Time Series*. Hoboken, NJ, USA: John Wiley & Sons, Inc., 2010. doi: https://doi.org/10.1002/9780470644560.
[27]
M. Ballings, D. Van den Poel, N. Hespeels, and R. Gryp, “Evaluating multiple classifiers for stock price direction prediction,” *Expert Systems with Applications*, vol. 42, no. 20, pp. 7046–7056, Nov. 2015, doi: https://doi.org/10.1016/j.eswa.2015.05.013.
[28]
J. Campbell, A. W. Lo, and A. Craig MacKinlay, “The Econometrics of Financial Markets,” *Macroeconomic Dynamics*, Jun. 2012, doi: https://doi.org/10.2307/j.ctt7skm5.
[29]
S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “Statistical and Machine Learning forecasting methods: Concerns and ways forward,” *PLOS ONE*, vol. 13, no. 3, p. e0194889, Mar. 2018, doi: https://doi.org/10.1371/journal.pone.0194889.
[30]
M. Lopez De Prado, *Advances in financial machine learning*. New Jersey: Wiley, 2018.
[31]
K. Kim, “Financial time series forecasting using support vector machines,” *Neurocomputing*, vol. 55, no. 1–2, pp. 307–319, Sep. 2003, doi: https://doi.org/10.1016/s0925-2312(03)00372-2.
[32]
A. C. Logue, “The Efficient Market Hypothesis and Its Critics,” *CFA Digest*, vol. 33, no. 4, pp. 40–41, Nov. 2003, doi: https://doi.org/10.2469/dig.v33.n4.1367.
[33]
N. Arora and P. M, “Financial Analysis: Stock Market Prediction Using Deep Learning Algorithms,” *SSRN Electronic Journal*, 2019, doi: https://doi.org/10.2139/ssrn.3358252.

10. Appendix

This section provides additional technical details supporting the implementation and evaluation of the stock prediction model.

Technology Stack

The project was developed using Python and widely adopted data science and machine learning libraries.

- **Programming Language:** Python
- **Libraries:** Pandas, NumPy, Matplotlib, Scikit-learn, TensorFlow/Keras
- **Data Source:** Yahoo Finance API (yfinance)
- **Development Environment:** Google Colab (**CPU:** Typically an Intel Xeon processor with 2 vCPUs, **RAM:** Approximately **12 GB, NVIDIA Tesla T4)**

Dataset Description

The dataset consists of historical daily stock data obtained from Yahoo Finance.

- **Stock used (final model):** Tesla (TSLA)
- **Time period:** Approximately 10 years
- **Frequency:** Daily

Main attributes:

- Date, Open, High, Low, Close, Adjusted Close, Volume

Feature Engineering

Several derived features were created to enhance model performance:

- Return (percentage price change)
- Moving Averages (MA10, MA20, MA50)
- Volatility (10-day rolling standard deviation)
- Volume Change

These features capture both **trend information and short-term market dynamics**.

Model Configuration

The final model (Phase 3) used the following settings:

- Lookback window: 60 days
- Train-test split: 80/20 (chronological)
- Scaling: MinMaxScaler (fit on training data only)
- Optimizer: Adam
- Loss function: Mean Squared Error (MSE)
- Batch size: 32
- Epochs: up to 30 with Early Stopping

Evaluation Metrics

Model performance was evaluated using both regression and classification metrics:

Regression:

- MAE
- RMSE
- R^2 Score

Classification:

- Directional Accuracy
- Precision
- Recall
- F1-score

Experimental Setup

All experiments were conducted using Google Colab with standard computational resources. The implementation ensured consistent preprocessing, training, and evaluation across all phases of the project.

Project Repository

The complete implementation of this project, including data processing scripts, model training code, and evaluation workflows, is available in the project's GitHub repository.

GitHub Repository Link:
https://github.com/Balkar-S/Stock-Market-Prediction-Model.git